

Tribhuvan University
Institute of Science and Technology
Nagarjuna College of Information Technology



Project Work Report

On

Sajilo Yatra: A Tour Management System

Under the Supervision of

Mr. Ramesh Singh Saud

Nagarjuna College of Information Technology

**In partial fulfillment of the requirements for the Bachelor's Degree in Computer Science and
Information Technology**

Submitted By:

Saugat Panta/ Roll No.79010868

Shovit Regmi/ Roll No. 79010869

Suman Basnet/ Roll No. 79010872

Submitted To:

Department of Computer Science and Information Technology

Nagarjuna College of Information Technology

Tribhuvan University

Institute of Science and Technology

Nagarjuna College of Information Technology



SUPERVISOR RECOMMENDATION LETTER

I hereby recommend that this project prepared under my supervision by SAUGAT PANTA, SHOVIK REGMI and SUMAN BASNET entitled “Sajilo Yatra” in partial fulfilment of the requirements for the degree of Bachelor in Computer Science and Information Technology be processed for final evaluation.

.....

Mr. Ramesh Singh Saud

Project Supervisor

CSIT Department

Nagarjuna College of IT

Tribhuvan University
Institute of Science and Technology
Nagarjuna College of Information Technology



CERTIFICATE OF APPROVAL

This is to certify that this project prepared by **SAUGAT PANTA, SHOVIT REGMI and SUMAN BASNET** entitled “Sajilo Yatra” in partial fulfilment of the requirement for the bachelor's degree in Computer Science and Information Technology has been well studied. In our opinion, it is satisfactory in the scope and quality as a project for the required degree.

.....
Mr. Ramesh Singh Saud

Supervisor

CSIT Department

Nagarjuna College of IT

.....
Internal Examiner

Nagarjuna College of IT

.....
Ms. Anita Joshi

Program Coordinator

CSIT Department

Nagarjuna College of IT

.....
External Examiner

Central Department of TU

ACKNOWLEDGEMENT

The success and outcome of this project required a lot of guidance and assistance from many people, and we feel extremely fortunate to have received this support throughout the completion of our final year project. Whatever we have accomplished is only due to such guidance and assistance, and we would not forget to thank them.

We would like to express our sincere gratitude to Nagarjuna College of Information Technology for providing us the environment, resources, and opportunity to carry out this project as part of our academic curriculum. We are especially thankful for the technical exposure and support that helped us apply our classroom learning to a real-world system.

We take this opportunity to express our profound gratitude and deep regards to our supervisor, Mr. Ramesh Singh Saud, HOD, Nagarjuna College of Information Technology, for his exemplary guidance, valuable feedback, and constant encouragement throughout the course of this project. His willingness to give his time so generously has been very much appreciated.

We would also like to thank our teachers and colleagues at the Department of Computer Science and Information Technology for their continuous support and constructive suggestions, which helped us refine both the technical design and implementation of Sajilo Yatra.

Finally, we would like to thank our friends and families for their patience and encouragement throughout this project. This project has been a valuable learning experience, allowing us to explore full-stack web development, algorithm design, and real-world system architecture in depth.

Saugat Panta (79010868)

Shovit Regmi (79010869)

Suman Basnet (79010872)

ABSTRACT

Most tour and trekking operators in Nepal still handle bookings the old way, phone calls, in-person visits, and messages on social media. This makes it hard for customers to browse available packages or book outside office hours, and hard for operators to track who booked what and whether payment has been received. Sajilo Yatra was built to fix this by putting browsing, booking, payment, and communication into one web platform.

The system has two sides. On the public side, users can browse tour packages, filter them by category, price, duration, difficulty, and rating, and search by keyword. On the admin side, the operator can add and edit packages, track bookings, verify payments, and view basic business analytics, all behind a JWT-protected login.

Search results are ranked using TF-IDF rather than plain keyword matching, so tours where a keyword matters in the description rank higher than ones that just mention it in passing. Each tour page also shows a "You Might Also Like" section, generated by a Cosine Similarity-based recommendation engine comparing tours by price, duration, difficulty, and category. Both were written from scratch in TypeScript, without any ML library.

The backend is built with NestJS and TypeORM on MySQL, while the frontend uses Next.js 15, Mantine UI, and Tailwind CSS. The booking flow supports Khalti, eSewa, and card payment options. Automated gateway processing was outside the implemented scope, so customers using local wallets upload a PDF receipt for manual verification by the admin. Booking confirmations and status changes trigger automated email notifications through Nodemailer.

Overall, Sajilo Yatra shows that a small tour operator's day-to-day work of listing packages, taking bookings, tracking payments, and keeping customers updated can run through one system instead of scattered spreadsheets and manual messages, while giving customers a smarter, faster way to find tours that suit them.

Keywords: *Tour Management System, Booking, TF-IDF, Cosine Similarity, Recommendation System, NestJS, Next.js, MySQL.*

TABLE OF CONTENTS

SUPERVISOR RECOMMENDATION LETTER.....	ii
CERTIFICATE OF APPROVAL.....	iii
ACKNOWLEDGEMENT	iv
ABSTRACT.....	v
LIST OF FIGURES	viii
LIST OF TABLES.....	ix
LIST OF ABBREVIATIONS.....	x
CHAPTER 1: INTRODUCTION.....	1
1.1 Introduction.....	1
1.2 Problem Statement.....	2
1.3 Objectives	3
1.4 Scope and Limitations.....	3
1.4.1 Scope.....	3
1.4.2 Limitations	4
1.5 Development Methodology	5
1.6 Report Organization.....	6
CHAPTER 2: BACKGROUND STUDY AND LITERATURE REVIEW.....	8
2.1 Background Study.....	8
2.1.1 Tour Management Systems.....	8
2.1.2 Web-Based Client-Server Architecture	8
2.1.3 Search Relevance Ranking (TF-IDF)	9
2.1.4 Content-Based Recommendation Engines (Cosine Similarity).....	9
2.1.5 Authentication with JWT	10
2.1.6 Digital Wallets in Nepal.....	10
2.2 Literature Review.....	10
2.2.1 Review of Existing Studies.....	11
2.2.2 Comparative Analysis.....	12
2.2.3 Research Gap	13
CHAPTER 3: SYSTEM ANALYSIS.....	15

3.1 System Analysis.....	15
3.1.1 Requirement Analysis.....	15
3.1.2 Feasibility Analysis.....	19
3.1.3. Analysis.....	21
CHAPTER 4: SYSTEM DESIGN.....	32
4.1 Design of the System	33
4.1.1 Refined Class Diagram	33
4.1.2 Refined Object Diagram	33
4.1.3 Refined State Diagram.....	34
4.1.4 Refined Sequence Diagram.....	34
4.1.5 Refined Activity Diagram.....	37
4.1.6 Components Diagram	38
4.1.7 Deployment Diagram.....	39
4.2 Algorithm Details.....	40
4.2.1 TF-IDF Search Relevance Ranking (Information Retrieval Algorithm).....	40
4.2.2 Content-Based Recommendation Engine (Cosine Similarity)	40
CHAPTER 5: IMPLEMENTATION AND TESTING	42
5.1 Implementation	42
5.1.1 Tools Used	42
5.1.2 Implementation Details of Modules.....	44
5.2 Testing.....	47
5.2.1 Unit Testing	47
5.2.2 System Testing.....	50
5.3 Result Analysis	53
CHAPTER 6: CONCLUSION AND FUTURE RECOMMENDATIONS	55
6.1 Conclusion	55
6.2 Future Recommendation.....	56
REFERENCES	58
APPENDICES	60
APPENDIX A: SCREENSHOTS.....	60
APPENDIX B: CODE SNIPPETS	75

LIST OF FIGURES

Figure 1.1: Iterative and Incremental Development Model.....	216
Figure 3.1: Use Case Diagram of Tour Management System	21
Figure 3.2: Gantt Chart	21
Figure 3.3: Class Diagram of Tour Management System	21
Figure 3.4: Object Diagram of Tour Management System.....	215
Figure 3.5: State Diagram – Booking Lifecycle of Tour Management System	21
Figure 3.6: Sequence Diagram - Customer Search of Tour Management System	21
Figure 3.7: Sequence Diagram – Customer Checkout of Tour Management System	21
Figure 3.8: Sequence Diagram – Admin Booking Status of Tour Management System	21
Figure 3.9: Activity Diagram of Tour Management System	21
Figure 4.1: Refined Class Diagram of Tour Management System.....	35
Figure 4.2: Refined Object Diagram of Tour Management System.....	35
Figure 4.3: Sequence Diagram - Customer Searches of Tour Management System.....	35
Figure 4.4: Sequence Diagram - Customer Searches of Tour Management System.....	35
Figure 4.5: Sequence Diagram - Customer Checkout of Tour Management System.....	36
Figure 4.6: Sequence Diagram (Admin Updates Booking) of Tour Management System	36
Figure 4.7: Refined Activity Diagram of Tour Management System	37
Figure 4.8: Component Diagram of Tour Management System.....	38
Figure 4.9: Deployment Diagram of Tour Management System	39

LIST OF TABLES

Table 2.1: Comparative Analysis of Case Studies.....	12
Table 3.1: Functional Requirements of Tour Management System	15
Table 3.2: Non-Functional Requirements of Tour Management System	18
Table 5.1: Tools and Technologies Used in Tour Management System	42
Table 5.2: Unit Test Cases of Tour Management System	48
Table 5.3: System Test Cases of Tour Management System.....	51

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
CSIT	Computer Science and Information Technology
CSS	Cascading Style Sheets
CSV	Comma-Separated Values
DB	Database
DTO	Data Transfer Object
HOD	Head of Department
HTML	Hyper Text Markup Language
HTTP	Hyper Text Transfer Protocol
HTTPS	Hyper Text Transfer Protocol Secure
IDF	Inverse Document Frequency
JSON	JavaScript Object Notation
JWT	JSON Web Token
MIME	Multipurpose Internet Mail Extensions

Abbreviation	Full Form
ORM	Object Relational Mapping
PDF	Portable Document Format
PK	Primary Key
RDS	Relational Database Service
REST	Representational State Transfer
ROI	Return on Investment
SDLC	Software Development Life Cycle
SQL	Structured Query Language
TF-IDF	Term Frequency–Inverse Document Frequency
TLS	Transport Layer Security
TU	Tribhuvan University
UI	User Interface
UML	Unified Modeling Language
URL	Uniform Resource Locator
UUID	Universally Unique Identifier

CHAPTER 1: INTRODUCTION

1.1 Introduction

Tourism is one of the fastest-growing industries in Nepal, attracting thousands of domestic and international travelers every year to its mountains, heritage sites, and natural landscapes. Managing and marketing tour packages efficiently has therefore become essential for tour operators. Traditional approaches such as manual telephone bookings, printed brochures, and spreadsheet-based tracking are increasingly inadequate: they are error-prone, difficult to scale, and offer no visibility to potential customers outside office hours.

Sajilo Yatra: Tour Management System is a full-stack, two-tier web application built to digitize and streamline the core workflows of a Nepal-based tour operator: package discovery, smart search and filtering, booking with optional add-ons, online receipt submission for local payment gateways, and post-booking communication via automated emails. The backend is implemented with NestJS (Node.js/TypeScript) and TypeORM over a MySQL relational database, exposing a REST API. The frontend is a Next.js 15 (App Router) application styled with Mantine UI and Tailwind CSS, consuming that API through Axios.

The platform provides two distinct user experiences: a public-facing tour catalogue where any visitor can browse, filter, search, and book tours; and a protected admin dashboard where the operator can manage packages, monitor bookings, update statuses, and view revenue analytics.

Rather than relying only on generic CRUD operations, two purpose-built algorithms enhance the system beyond a simple database front-end: a Content-Based Recommendation Engine using Cosine Similarity that powers the "You Might Also Like" feature on every tour detail page, and a TF-IDF Search Relevance Ranking engine that scores and orders tour results for keyword searches. Both algorithms are implemented in plain TypeScript inside the NestJS backend with no external machine-learning dependencies.

The checkout flow supports three payment methods: Khalti, eSewa, and Card. For the two local digital-wallet options, customers upload a PDF payment receipt which is stored server-side and reviewed by the admin before the booking is confirmed. Automated transactional

emails for booking confirmation and status-updated notifications are dispatched through a Nodemailer-backed mail service.

Therefore, the Tour Management System aims to provide a secure, modern, and efficient platform that brings package discovery, intelligent search, booking management, payment tracking, and customer communication together under one system, with role-based access and algorithm-backed tour recommendations and ranking.

1.2 Problem Statement

Nepal's tour and trekking operators traditionally rely on offline channels phone calls, in-person visits, and social media messages to receive booking inquiries. This approach makes it difficult for customers to browse available packages, compare prices and durations, or confirm a booking outside business hours. Operators themselves face the burden of manually recording booking details, tracking payment statuses, and notifying customers individually, all of which are time-consuming and error-prone.

On the discovery side, simple text matching (SQL LIKE) used by basic systems fails to rank results by relevance: a tour whose name barely mentions a keyword is listed identically to one whose description is rich with that term. Similarly, without a recommendation engine, a customer viewing one tour receives no meaningful suggestion of related tours based on attributes like price range, difficulty, duration, and category reducing the chance of additional bookings.

Admins lack a centralized view of their business performance. Revenue trends, booking statuses, popular packages, and customer records are scattered across spreadsheets, making reporting slow. There is also no automated mechanism to send booking confirmations or status-change notifications to customers, requiring staff to communicate manually.

Although Khalti and eSewa provide merchant-integration services, automated gateway processing was not implemented within this project's scope. The system therefore records the selected payment method and allows customers to upload a screenshot or PDF receipt for the administrator to review against the corresponding booking.

Therefore, there is a need for a secure, centralized web platform that enables customers to discover and book tours online, supports local payment receipt workflows, automates customer communication, provides admin analytics and booking management, and enhances the tour discovery experience with intelligent search ranking and content-based recommendations.

1.3 Objectives

- To develop a web-based Tour Management System that allows customers to discover, filter, search, and book tour packages online, using TF-IDF search ranking and a Cosine Similarity-based recommendation engine.
- To build a multi-step checkout system supporting Khalti, eSewa, and Card payments, with PDF receipt upload and automated email notifications for booking confirmations and status updates.
- To provide a secure, JWT-protected admin dashboard for managing tour packages, monitoring bookings, verifying payments, and viewing revenue and booking analytics.

1.4 Scope and Limitations

1.4.1 Scope

- Public users can browse the full tour catalogue and apply filters by category, difficulty, price range, duration, rating, family-friendly, and free cancellation.
- Customers can search tours by keyword, with results ranked using TF-IDF relevance scoring.
- Each tour detail page displays itinerary, highlights, inclusions/exclusions, photo gallery, and a "You Might Also Like" section powered by Cosine Similarity.
- Customers can complete a multi-step booking with traveler details and optional add-ons.

- Customers can select a payment method (Khalti, eSewa, or Card) and upload a PDF receipt for wallet payments.
- Customers automatically receive email notifications for booking confirmation and status updates.
- Admins can create, edit, activate/deactivate, and delete tour packages, including itinerary days, highlights, inclusions, add-ons, and images.
- Admins can view all bookings, update booking status, and review or download uploaded payment receipts.
- Admins can access a dashboard with revenue charts, booking statistics, and popular-tour analytics.
- Admin routes are protected using JWT-based authentication.

1.4.2 Limitations

- The system is designed for a single tour operator; multi-tenant or multi-company support is not implemented.
- Real payment-gateway integration (Khalti API, eSewa API, Stripe) is not implemented; payments are verified manually by the admin through uploaded receipts.
- There is no mobile application; the system is web-based only, accessed through a browser.
- Customer review and rating submission through the platform is not implemented; ratings are seeded/managed by the admin.
- Real-time notifications (WebSocket/push) are not implemented; status updates reach customers via email only.
- The recommendation engine uses content-based filtering and does not incorporate collaborative filtering (user behavior data).
- Internet connectivity and a modern browser are required to access the platform.

- Database schema synchronization is used for local development; production deployments should switch to versioned TypeORM migrations.

1.5 Development Methodology

This project followed an Iterative and Incremental Development Methodology, in which the system was built in multiple stages instead of all at once. Each increment added a working, testable piece of functionality on top of what was already built, so the system was never in a completely broken state for long and problems could be caught early rather than at the end.

We started with requirement analysis and system design, working out the database schema, the REST API structure, and how the two algorithms would fit into the package service before writing any real code. From there, the backend was set up first using NestJS, TypeORM, and MySQL, alongside the Next.js frontend shell. Once the core skeleton was in place, we built out the authentication and package management modules, since almost everything else in the system depends on having users and packages to work with. The booking and checkout flow came next, including traveler details, optional add-ons, and the three payment methods, since this is the core transaction the whole platform exists to support.

With the main booking flow working, we moved on to the two algorithms that set this project apart from a plain CRUD system: TF-IDF for search relevance and Cosine Similarity for the recommendation engine, both of which were built and tested independently before being wired into the tour catalogue and detail pages. The admin dashboard, the mail service for automated emails, and revenue analytics were added in the final increments, once the customer-facing side was stable enough to generate real data to display.

Each increment went through the same short cycle of coding, testing, debugging, and refining before being merged into the growing system. This kept individual modules easier to reason about and made it possible to catch integration issues (for example, a booking status change not triggering the right email) early, instead of only discovering them once the whole system was assembled.

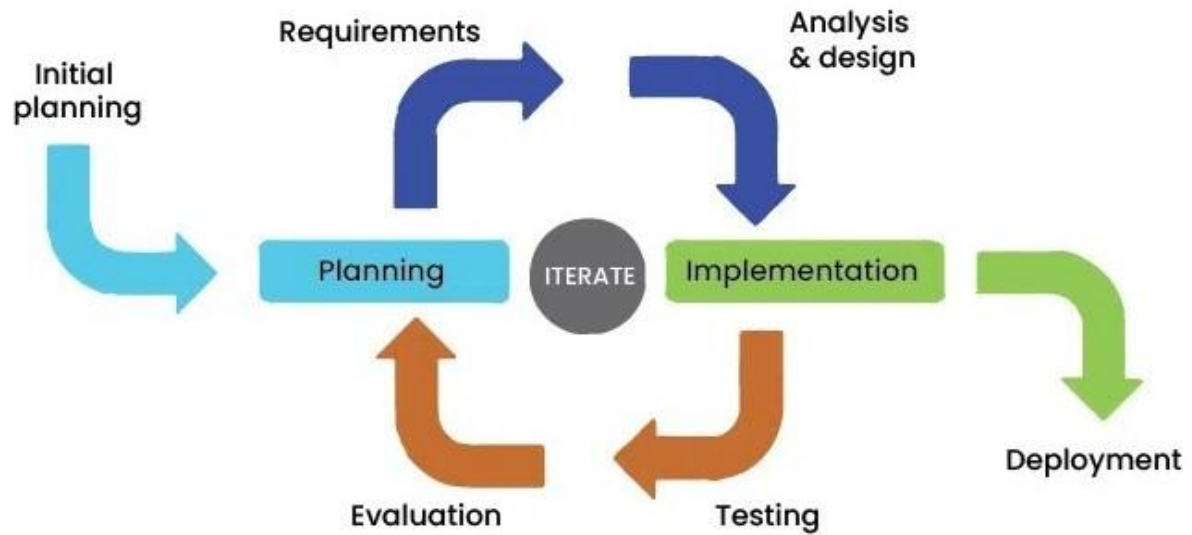


Figure 1.1: Iterative and Incremental Development Model

1.6 Report Organization

This report is organized into different chapters describing various aspects of the project.

- **Chapter 1** introduces the project background, problem statements, objectives, scope and limitations, development methodology, and report organization.
- **Chapter 2** discusses the theoretical background, related technologies, existing systems, and previous concepts related to tour/travel management systems, information retrieval, and recommendation algorithms.
- **Chapter 3** includes requirement analysis, functional and non-functional requirements, feasibility analysis, and object-oriented analysis (class, object, state, sequence, and activity diagrams) of the proposed system.
- **Chapter 4** describes the architectural design, database design, and interface design of the system.
- **Chapter 5** explains the implementation details, technologies used, algorithm integration, testing procedures, and system outputs.

- **Chapter 6** summarizes the overall project, achievements, limitations, and possible future enhancements of the system.

CHAPTER 2: BACKGROUND STUDY AND LITERATURE REVIEW

2.1 Background Study

2.1.1 Tour Management Systems

A tour management system is a web-based platform that brings together tour discovery, booking, and business administration in one place, instead of leaving them spread across phone calls, spreadsheets, and social media messages. For a small operator, this usually means solving two problems at once: giving customers an easy way to find and book what they want, and giving the operator a single dashboard to see what's actually happening in their business. Most academic and industry work in this space agrees on the same basic building blocks, a package catalogue, a booking workflow, and some form of payment tracking, even when the specific technology stack differs. Abu-AlShaeer et al. [1] describe a similar centralized tourism platform built to replace manual, error-prone tour and hotel booking processes. They report that automating these workflows reduced errors and improved both booking speed and the operator's visibility into their own business. Sajilo Yatra follows the same underlying idea, but focuses specifically on the workflow of a Nepal-based tour operator, including local payment methods that most generic tourism platforms don't need to account for.

2.1.2 Web-Based Client-Server Architecture

A web-based system like this one follows a client-server architecture, where the browser (client) sends requests over HTTP and a backend server processes them and returns data, usually as JSON. Sajilo Yatra splits this into two separate applications, a NestJS backend that exposes a REST API, and a Next.js frontend that consumes it. Keeping these separate rather than building one monolithic application makes it easier to develop, test, and deploy each side independently, and is a common pattern in modern web development.

NestJS is a Node.js framework, written in TypeScript, that organizes backend code into modules, controllers, and providers, and pairs naturally with an ORM such as TypeORM to map TypeScript classes onto MySQL tables. The choice of Node.js as the runtime is not just a

matter of convenience. Odeniran et al. [2] compared Node.js and PHP directly using identical sorting and searching workloads and found that Node.js consistently outperformed PHP in latency, which supports its use here, where search and recommendation queries need to feel fast to the end user.

2.1.3 Search Relevance Ranking (TF-IDF)

Basic keyword search, the kind that matches a query against a database using something like SQL's LIKE, has no real concept of relevance. A tour whose name barely mentions a keyword gets treated the same as one whose entire description is built around it. Term Frequency to Inverse Document Frequency (TF-IDF) is one of the foundational techniques in information retrieval used to fix exactly this problem, scoring how important a term is to a specific document relative to how common it is across the whole collection. Ramos [3] demonstrated that even this comparatively simple algorithm is effective at surfacing the words in a document collection that genuinely distinguishes one document from another. It remains one of the most widely used ranking techniques in search systems today because it is fast, easy to reason about, and needs no training data or machine learning infrastructure. That makes it a good fit for Sajilo Yatra, where each "document" is simply a tour's name, description, category, and other text fields, with no external ML library required.

2.1.4 Content-Based Recommendation Engines (Cosine Similarity)

Recommendation systems generally fall into two broad categories: collaborative filtering, which recommends items based on what similar users liked, and content-based filtering, which recommends items based on their own attributes. Collaborative filtering needs a reasonable amount of user interaction history to work well, which makes it a poor fit for a new platform or a newly listed tour with no bookings yet, a problem generally known as the "cold-start" problem. Content-based filtering avoids this since it only needs the item's own attributes, not user history.

Cosine similarity is the standard mathematical technique behind most content-based recommenders. It measures the angle between two feature vectors rather than their raw distance, which makes it less sensitive to differences in scale between features. Sarwar et al. [4], in one of the foundational studies on recommendation algorithms for e-commerce, showed

that similarity-based methods can meaningfully improve the relevance of suggested items compared to naive approaches. More recently, Munkholm et al. [5] implemented a cosine-similarity-based recommender for an e-commerce home décor platform and found it effective and computationally lightweight, surfacing relevant items without needing a large volume of historical interaction data. Their finding is a big part of why Sajilo Yatra's "You Might Also Like" feature works the way it does, comparing tours by price, duration, difficulty, and category rather than relying on booking history.

2.1.5 Authentication with JWT

Authentication and authorization are two different concerns: authentication confirms who a user is, while authorization decides what they're allowed to do. Sajilo Yatra uses JSON Web Tokens (JWT) to protect its admin routes, which is a compact, signed token issued at login and sent with every subsequent request instead of relying on server-side session storage. Dalimunthe et al. [6] evaluated JWT-based REST API authentication combined with HMAC-SHA512 signing and found it to provide a scalable and secure approach to session management, particularly well suited to stateless REST APIs like the one this project exposes. This stateless property is part of why JWT was chosen here, it keeps the backend simple to scale, since no session state needs to be shared or synchronized across server instances.

2.1.6 Digital Wallets in Nepal

Khalti and eSewa are the two dominant digital wallets used for online payments in Nepal, and neither offers the kind of direct, developer-friendly payment gateway API that small operators can easily integrate into a custom system. Acharya [7] documents how eSewa and Khalti have driven the growth of digital financial habits in Nepal since their respective launches, while also noting that many smaller platforms still rely on manual verification of payment proof rather than automated gateway integration. That matches the design of Sajilo Yatra's checkout flow, where customers upload a PDF receipt for Khalti and eSewa payments and the admin reviews and confirms it manually, a practical compromise for a small operator without the infrastructure or volume to justify a full payment gateway integration.

2.2 Literature Review

This section reviews seven studies relevant to Sajilo Yatra, covering tourism management systems, search relevance ranking, content-based recommendation, backend framework selection, authentication, and digital payments in Nepal.

2.2.1 Review of Existing Studies

Tourism and Booking Platforms.

Abu-AlShaeer et al. [1] built a centralized online tourism management system to replace manual booking channels, using a PHP and SQL Server stack. Their system covered package browsing, booking, and admin-side package management, and the study reported that centralizing these workflows improved both operational efficiency and the accuracy of booking records compared to manual tracking. Their system relies on plain database queries for search, though, and includes no relevance ranking or recommendation capability. Customers simply see results in whatever order the database happens to return them.

Search Relevance.

Ramos [3] established the core case for TF-IDF as a lightweight, effective method for scoring word relevance within a document collection, without requiring any training data. The work predates modern web applications, but its central finding, that TF-IDF reliably distinguishes distinctive terms from common ones, remains the basis for how TF-IDF is applied across countless later search systems, including this one.

Content-Based Recommendation.

Sarwar et al. [4] compared recommendation algorithms for e-commerce and demonstrated that similarity-based techniques improve recommendation quality over simple heuristics. Munkholm et al. [5] went further and applied cosine similarity specifically to a content-based recommender for an e-commerce platform, confirming it works well even with a relatively small, attribute-rich dataset, which closely matches the scale and structure of Sajilo Yatra's own tour catalogue.

Backend Framework Choice.

Odeniran et al. [2] benchmarked Node.js against PHP using identical algorithmic workloads and found Node.js to be measurably faster, supporting the choice of a Node.js-based framework, NestJS, for this project's backend.

Authentication.

Dalimunthe et al. [6] evaluated JWT-based authentication for REST APIs and found it secure and well suited to stateless service architectures, directly supporting its use for protecting Sajilo Yatra's admin routes.

Digital Payments in Nepal.

Acharya [7] traced the growth of Khalti and eSewa in the Nepali market and noted that many smaller businesses still depend on manual receipt verification rather than automated gateway integration, a gap that Sajilo Yatra's receipt-upload workflow was designed to work within rather than around.

2.2.2 Comparative Analysis

Table 2.1: Comparative Analysis of Case Studies

Ref .	Author(s) & Year	Research Objective	Method	Key Findings	Limitations
[1]	Abu-AlShaeer et al.	Centralize tourism booking	PHP, SQL Server	Improved efficiency & accuracy	No search ranking or recommendations
[2]	Odeniran et al.	Compare backend performance	Node.js vs PHP	Node.js faster on identical workloads	Focused on raw algorithm speed only

[3]	Ramos	Determine word relevance	TF-IDF	Effective, lightweight relevance scoring	No semantic understanding of terms
[4]	Sarwar et al.	Improve e-commerce recommendations	Similarity-based filtering	Similarity methods outperform naive approaches	Predates modern web-scale datasets
[5]	Munkholm et al.	Recommend items by content	Cosine similarity	Effective with small, attribute-rich data	Doesn't use collaborative signals
[6]	Dalimunthe et al.	Secure REST API sessions	JWT + HMAC-SHA512	Secure, scalable, stateless	Token storage security not addressed
[7]	Acharya	Study digital wallet adoption	Survey/analyses	Khalti/eSewa dominate, manual verification common	Nepal-specific, not global

2.2.3 Research Gap

Looking at these studies together, a few gaps stand out that directly shaped how Sajilo Yatra was designed.

First, tourism management systems in the existing literature, such as the one built by Abu-AlShaeer et al. [1], tend to focus on digitizing the booking workflow itself without addressing

how customers actually find the right tour among many options. Search and recommendation are usually treated as separate research problems from booking systems entirely, when in practice they belong in the same product. Sajilo Yatra brings both together, using TF-IDF and cosine similarity directly inside the same platform that handles booking and payment.

Second, none of the reviewed tourism or recommendation studies were built with Nepal's specific payment landscape in mind. Digital wallets like Khalti and eSewa don't offer the kind of gateway APIs that Western or larger-scale systems typically assume are available, something most generic bookingsystem literature doesn't account for. Acharya's work [7] confirms that manual receipt verification remains common practice, which is exactly the workflow Sajilo Yatra was built around rather than forcing an unavailable gateway integration.

Third, the recommendation and search literature reviewed here [3][4][5] is largely domain-agnostic, covering home décor, general e-commerce, and text corpora, rather than tours specifically, which carry their own mix of attributes such as difficulty, duration, season, and group size that differ from typical retail products. Sajilo Yatra adapts these general-purpose techniques to the specific structure of tour data.

Finally, while Dalimunthe et al. [6] and Odeniran et al. [2] support the individual technology choices behind this project (JWT authentication, Node.js backend), no single study combines all of these pieces, TF-IDF search, cosine similarity recommendations, JWT-secured admin access, and Nepal-specific payment handling, into one system. Sajilo Yatra's contribution is bringing these individually well-supported techniques together into a single, coherent platform built for a real operational context.

CHAPTER 3: SYSTEM ANALYSIS

3.1 System Analysis

System analysis is the detailed study of the requirement, feasibility, and design model of the proposed system. This includes identifying what functions the system is required to perform (functional requirements), stating the quality attributes of the system (non-functional requirements), performing a feasibility study from technical, operational, economic, and schedule points of view, and building models of the system using appropriate diagrams.

3.1.1 Requirement Analysis

i. Functional Requirements

The functional requirements define the exact behaviors and functions that the system must execute. Use Case Diagrams are used to depict them.

Table 3.1: Functional Requirements of Tour Management System

Requirement	Description
Tour Catalogue & Filtering	Public users can browse all active tour packages and apply filters by category, difficulty, price range, duration, rating, family-friendly flag, and free-cancellation flag.
TF-IDF Keyword Search	Keyword searches rank results by TF-IDF relevance score (name, description, location, category, difficulty, tagline) rather than a default price sort.
Tour Detail Page	Each tour displays a hero image, photo gallery, highlights, day-by-day itinerary, inclusions/exclusions, available add-ons, rating, and booking sidebar.
Cosine Similarity Recommendations	The detail page surfaces up to 3 "You Might Also Like" tours ranked by feature-vector cosine similarity.

Requirement	Description
Multi-Step Checkout	A three-step booking form collects traveler personal details, optional add-ons, departure date/notes, payment method, and (for Khalti/eSewa) a PDF receipt upload.
Booking Management (Admin)	Admin can view all bookings in a table, update status (pending/confirmed/cancelled), view and download receipt PDFs, and resend confirmation emails.
Package Management (Admin)	Admin can create, edit, activate/deactivate, and delete tour packages, including itinerary days, highlights, inclusions, add-ons, and images.
Automated Emails	The system sends an HTML booking-confirmation email on creation and a status-update email whenever the admin changes a booking's status.
Contact Form	A public contact page stores name, email, subject, and message; the admin can view messages in an inbox and mark them read.
Admin Dashboard Analytics	Stats grid (revenue, bookings, active tours, pending bookings), revenue chart (monthly), popular-tours table, customer table, and reviews list.
JWT Authentication	Secure admin login; protected routes reject requests without a valid JWT; Next.js middleware redirects unauthenticated users away from admin pages.

Use Case Diagram

The following use case diagram illustrates the interactions between the main actors (Public User and Admin) and the system's core functionalities.

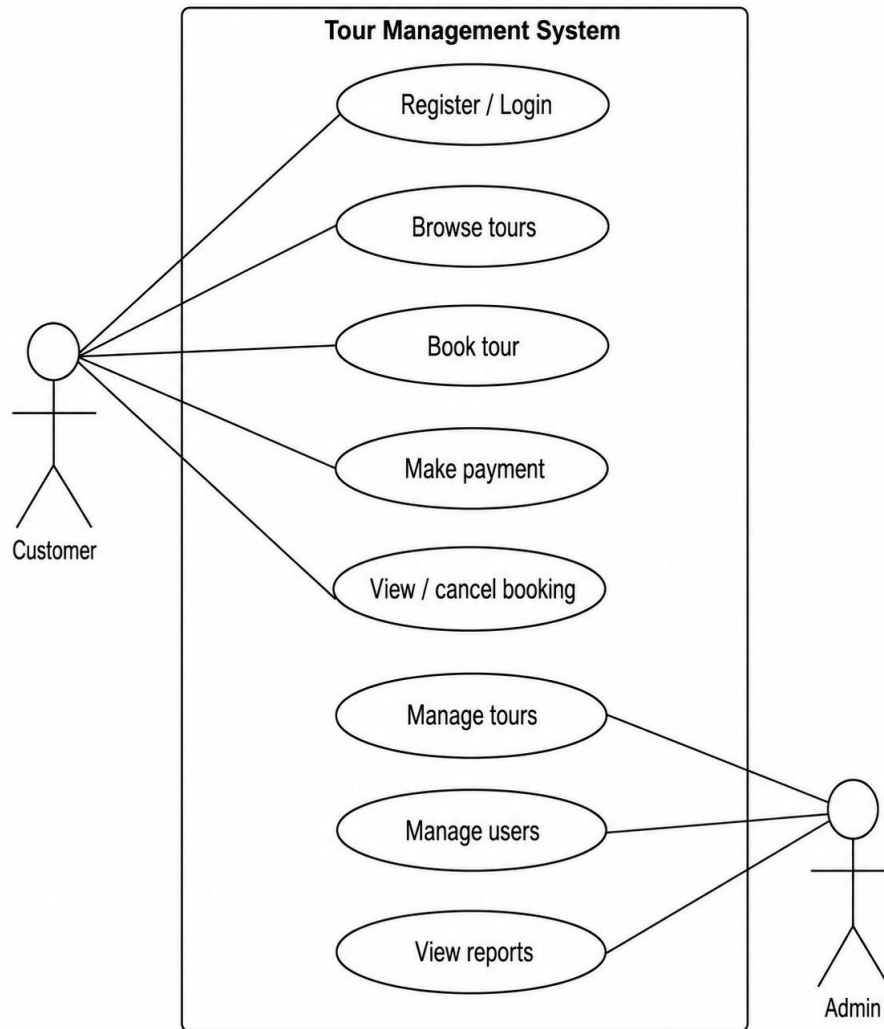


Figure 3.1: Use Case Diagram of Tour Management System

ii. Non-Functional Requirements

Non-Functional Requirements are the quality and performance criteria that the system shall meet. They include security, scalability, usability, reliability, maintainability, and system efficiency.

Table 3.2: Non-Functional Requirements of Tour Management System

Requirement	Description
Security	JWT authentication protects admin routes; passwords are hashed with bcrypt; the password column uses select: false so it is never returned in queries; receipt files are stored on the server's filesystem and not publicly accessible by path.
Scalability	The modular NestJS architecture (one module per domain: packages, bookings, contacts, auth, mail) is designed to accommodate a growing package catalogue and booking volume without restructuring the codebase.
Usability	The public catalogue and admin dashboard provide clean, role-specific interfaces: filters and a search bar for customers; tables, modals, and charts for the admin.
Reliability	Email failures are caught and logged without crashing the booking flow; file-upload validation rejects non-PDF files and files exceeding 10 MB.
Performance	TF-IDF and Cosine Similarity operate in $O(n \cdot L)$ and $O(n \cdot d)$ respectively over the filtered catalogue; linear in catalogue size, giving acceptable response times for typical inventories.
Maintainability	Clearly separated modules (auth, packages, bookings, contacts, mail) on the backend and a component-based Next.js frontend make the system straightforward to extend.
Interoperability	A single REST API with a consistent JSON response contract serves both the Next.js frontend and any future client (mobile app, third-party integration).

Requirement	Description
Availability	The system is accessible via any modern web browser with internet connectivity, deployed on standard Node.js-compatible hosting.

3.1.2 Feasibility Analysis

i. Technical Feasibility:

Sajilo Yatra's technical feasibility rests on mature, well-documented tools rather than experimental ones. NestJS, TypeORM, and MySQL form a modular, type-safe backend well suited to the relational structure of packages and bookings, while Next.js 15 with Mantine UI and Tailwind CSS gave the frontend most of its components, including tables, modals, forms, and charts, without building a design system from scratch. JWT (via `@nestjs/jwt` and `passport-jwt`) keeps admin authentication stateless, and `class-validator/class-transformer` enforce DTO-level validation before requests reach business logic. The two algorithms that set this project apart, TF-IDF search ranking and Cosine Similarity recommendations, were written as plain TypeScript utilities rather than pulled from an ML library, keeping every ranked result explainable. Multer handles receipt uploads with MIME-type and size validation, and Nodemailer (via `@nestjs-modules/mailer`) handles transactional email. None of these required paid licensing or specialized infrastructure, which kept the stack realistic for a small academic project.

ii. Operational Feasibility:

Operationally, the system needed to work for both the customer browsing tours and the operator running the business. Customers can browse, filter, view recommendations, complete a multi-step booking, and upload a receipt without an account or any technical familiarity. Admins manage the package catalogue, review bookings, verify receipts, and check revenue trends from one dashboard, replacing spreadsheets and phone-based tracking. Because TF-IDF and Cosine Similarity run automatically, package discovery

improves without any manual curation. The admin manages packages as usual, and the system handles ranking on top.

iii. Economic Feasibility:

The system avoids the costs that typically put custom software out of reach for a small operator. Costs are limited to developer time, standard Node.js-compatible hosting, and a single MySQL instance plus modest storage for receipt PDFs, with no specialized infrastructure required. In return, the operator gains less manual booking handling, 24/7 availability, automated communication, and centralized revenue data instead of scattered spreadsheets. Since the system replaces existing manual work rather than adding new hardware or paid services, direct operator funding is realistic, and the resulting data becomes a reusable asset for reporting and marketing.

iv. Schedule Feasibility:

The project ran roughly three months, January to late March, across six phases. Planning (January) covered the master schedule and core modules. Analysis (late Jan to early Feb) translated the manual booking workflow into concrete requirements and algorithm specs. Design (February) covered architecture, database schema, API structure, and wireframes. Implementation (late Feb to March), the longest phase, built the backend and frontend and integrated TF-IDF and Cosine Similarity once the booking flow was stable. Testing (March) covered unit, integration, and end-to-end system testing, including algorithm correctness. Deployment (late March) covered hosting setup and release. Figure 3 shows how these phases overlapped across the three-month window.

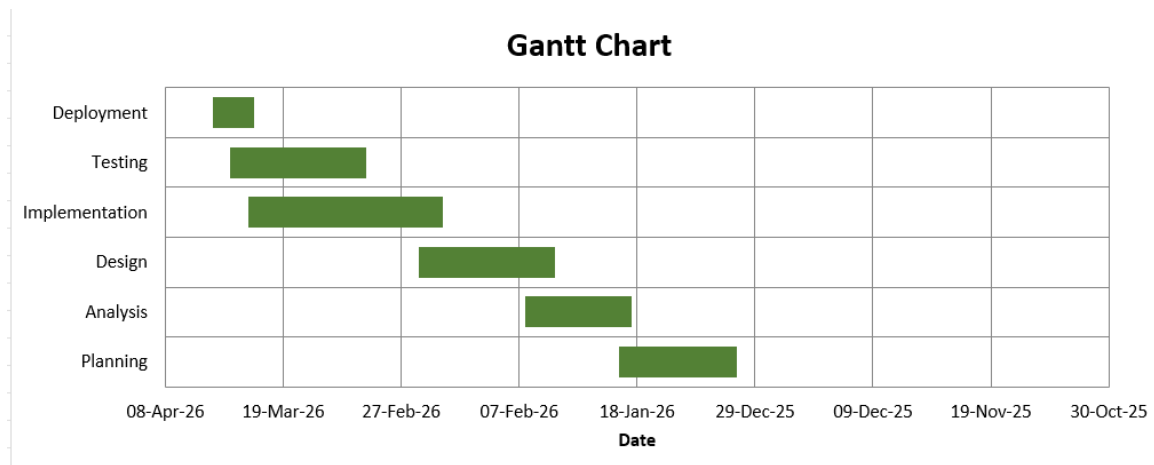


Figure 2.2: Gantt Chart

3.1.3. Analysis

Object Oriented Approach:

The Tour Management System follows an object-oriented approach; the system is modeled using Unified Modeling Language (UML) diagrams instead of structured tools alone. The static structure of the system is represented using Class and Object Diagrams, the dynamic behavior of individual objects using State Diagrams and Sequence Diagrams, and the overall business process flow across actors using an Activity Diagram.

Object Modeling:

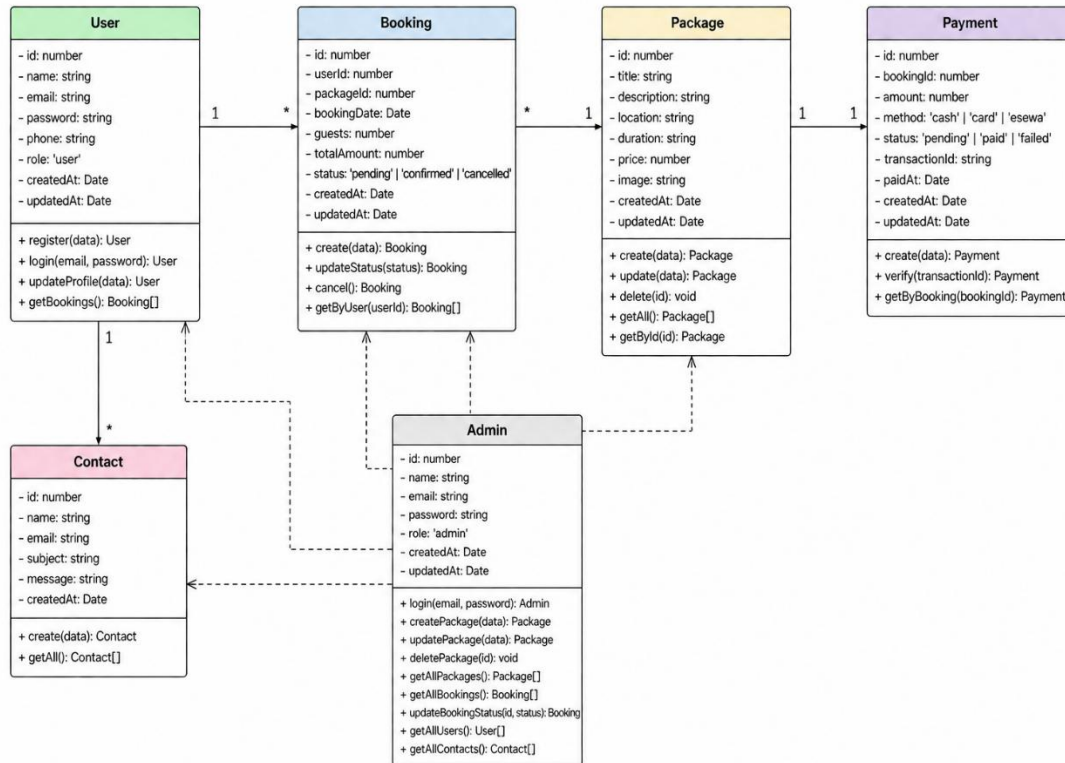


Figure 3.3: Class Diagram of Tour Management System

Figure 3.3 illustrates the analysis class diagram of the Sajilo Yatra system. The figure summarizes the core classes shown: User, Admin, Contact, Booking, Package, and Payment. Package represents the tour offering and is linked to booking and payment records. User and Admin support customer and administrative interactions, while Contact stores customer inquiries. The diagram presents the principle attributes, operations, and relationships used during analysis. Additional supporting package classes such as TourItinerary, TourHighlight, TourInclusion, TourAddon, and TourImage are described below and refined in Chapter 4.

Class Descriptions:

Package

Represents a tour package offered by the operator. Contains all listing information, pricing, and associated sub-entities.

Attributes: id (PK), name, slug, description, price, days, location, difficulty, category, rating, image, badge, tagline, reviewCount, isActive, freeCancellation, familyFriendly, hasDeal, isNew, createdAt.

Booking

Records a customer's reservation for a specific package, including traveler details, payment method, add-ons, and receipt path.

Attributes: id (PK), tourId (ref. Package), firstName, lastName, email, phone, country, travelers, departureDate, notes, paymentMethod, tourPrice, addonsTotal, totalAmount, selectedAddons (JSON), status, receiptPath, createdAt, updatedAt.

User

Represents an admin account used to access the protected dashboard.

Attributes: id (UUID PK), name, email, password (hashed), role (user/admin), isActive, createdAt, updatedAt.

Contact

Stores an inquiry submitted through the public contact form.

Attributes: id (PK), name, email, subject, message, read, createdAt.

TourItinerary

A single day entry in a package's day-by-day itinerary.

Attributes: id (PK), packageId (ref. Package), day, title, description.

TourHighlight

A key highlight point of a tour package (icon, title, description).

Attributes: id (PK), packageId (ref. Package), icon, title, description.

TourInclusion

An inclusion or exclusion item listed on the tour detail page.

Attributes: id (PK), packageId (ref. Package), text, type (include/exclude).

TourAddon

An optional add-on service that can be selected during checkout.

Attributes: id (PK), packageId (ref. Package), name, description, price.

TourImage

An additional gallery image associated with a tour package.

Attributes: id (PK), packageId (ref. Package), url, altText.

AlgorithmModule

Represents the two shared algorithms implemented in the package service.

Methods: rankByTfIdf(packages, keyword), recommendByCosine(packages, targetId, limit)

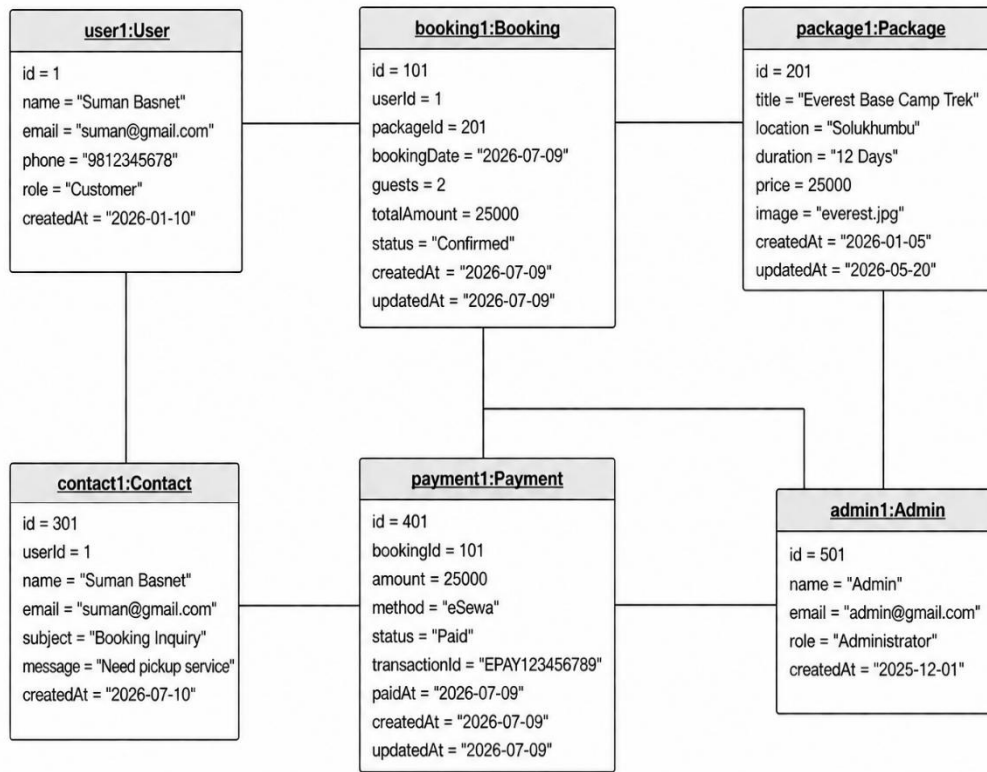


Figure 3.4: Object Diagram of Tour Management System

Figure 3.4 illustrates the analysis object diagram of the Sajilo Yatra system. It provides a snapshot of the system at a particular point in time by showing instances of the primary domain classes and their relationships, for example, a Package instance for "Everest Base Camp Trek" linked to a newly created Booking instance carrying a specific customer's firstName, travelers, and status values. The diagram demonstrates how objects interact conceptually during system execution, immediately after a customer submits a booking and the system dispatches a confirmation email. Only the essential object information is presented to represent the system from an analysis perspective without implementationspecific details

Dynamic Modeling

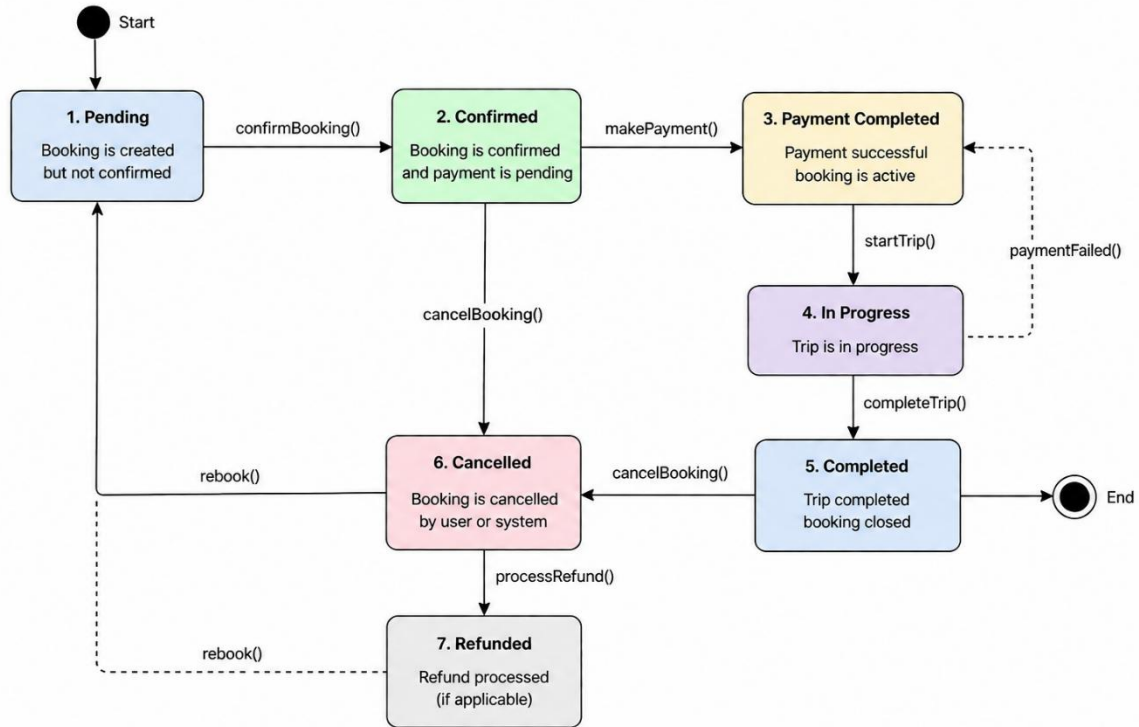


Figure 3.5: State Diagram - Booking Lifecycle of Tour Management System

Figure 3.5 illustrates the analysis state diagram of the booking lifecycle. It shows the major states of a Booking object, moving through Pending, Confirmed, and Cancelled as the customer submits a reservation and the admin reviews the uploaded payment receipt. The diagram presents the conceptual lifecycle of the booking without implementation-specific details.

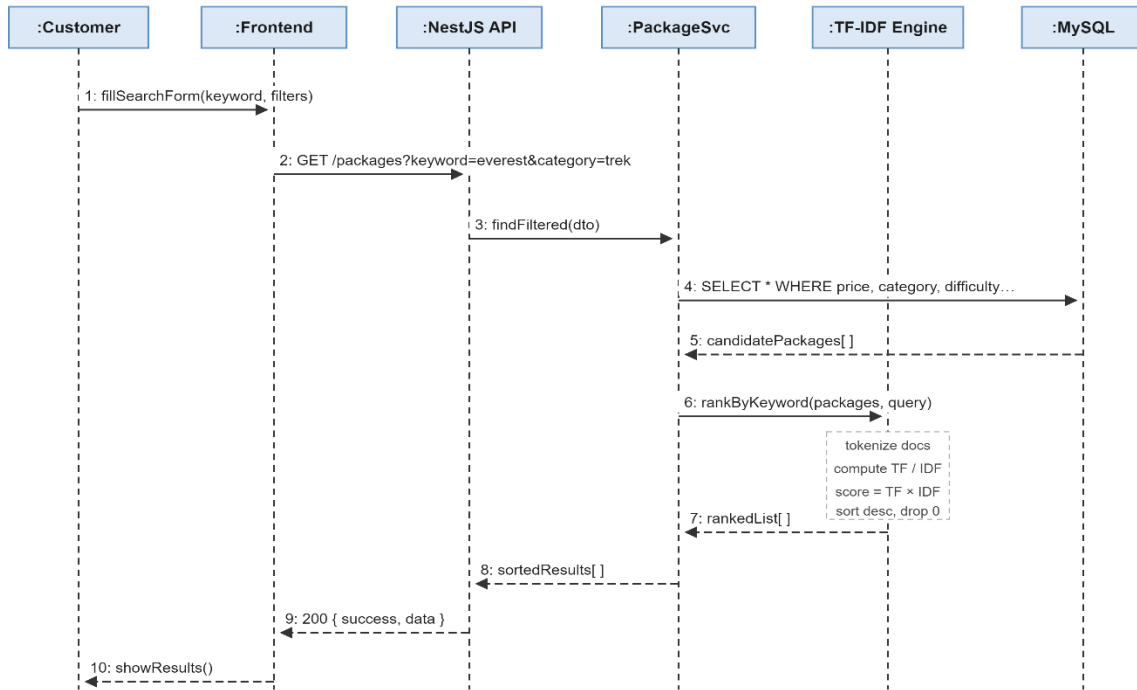


Figure 3.6: Sequence Diagram - Customer Search of Tour Management System

Figure 3.6 illustrates the analysis sequence diagram for the customer search use case. It presents the high-level interaction between the customer and the Sajilo Yatra system, beginning with a keyword query and ending with the Package results being ranked by the TF-IDF relevance score and displayed. The diagram focuses on the overall interaction flow without implementation details.

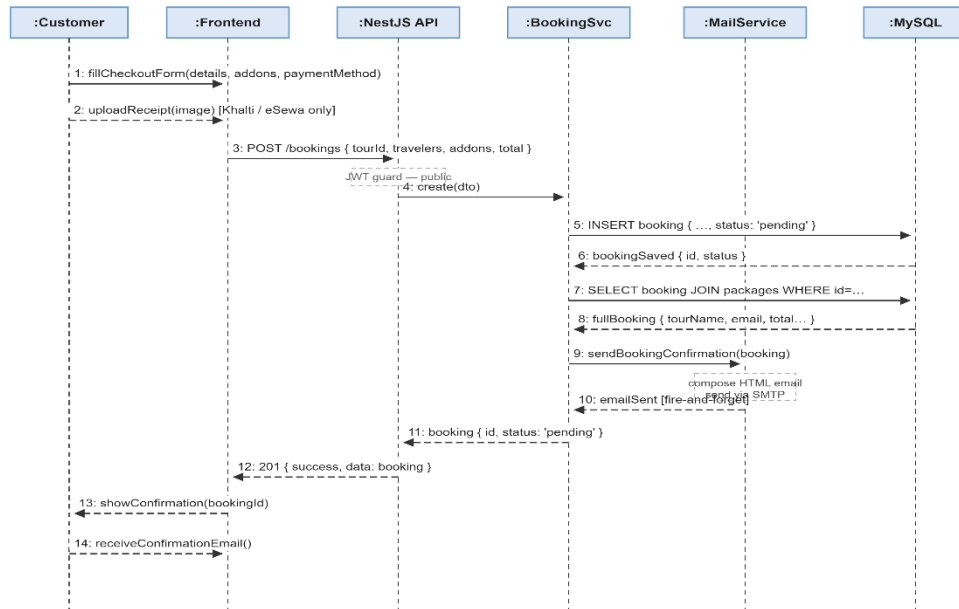


Figure 3.7: Sequence Diagram - Customer Checkout of Tour Management System

Figure 3.7 illustrates the analysis sequence diagram for the checkout use case. It presents the high-level interaction between the customer and the system, beginning with traveler-detail entry and payment-method selection and ending with a new Booking record being created and a PDF receipt stored against it. The diagram focuses on the overall interaction flow without implementation details.

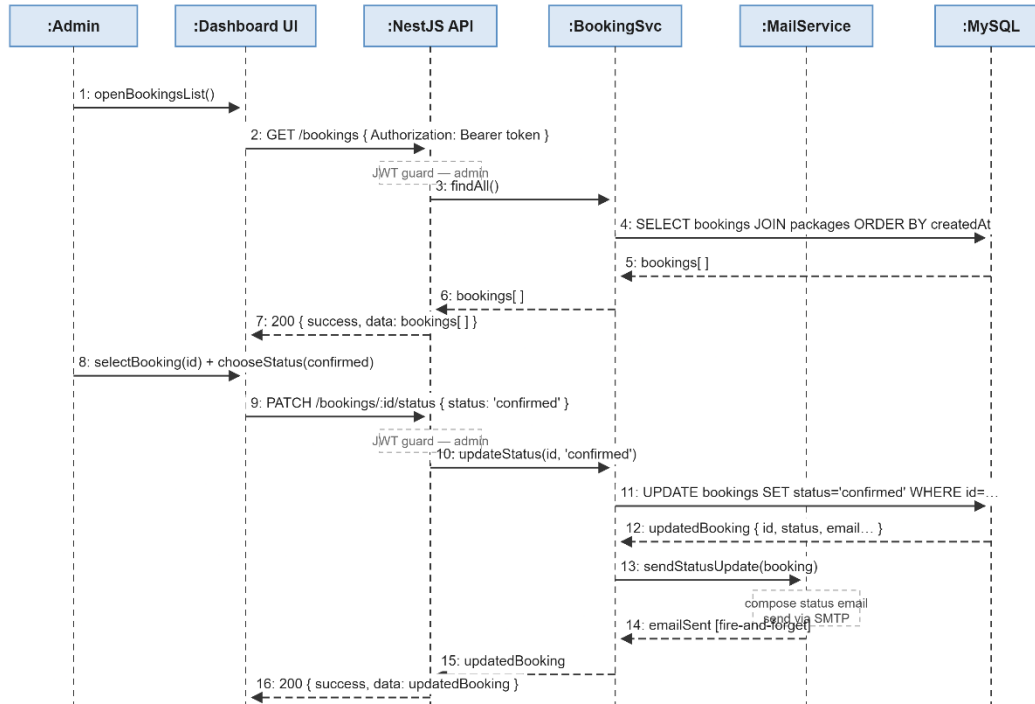


Figure 3.8: Sequence Diagram - Admin Booking Status of Tour Management System

Figure 3.8 illustrates the analysis sequence diagram for the booking-status use case. It presents the high-level interaction between the admin and the system, beginning with the admin reviewing a Booking's uploaded receipt and ending with the booking status being updated and a confirmation email dispatched to the customer. The diagram focuses on the overall interaction flow without implementation details.

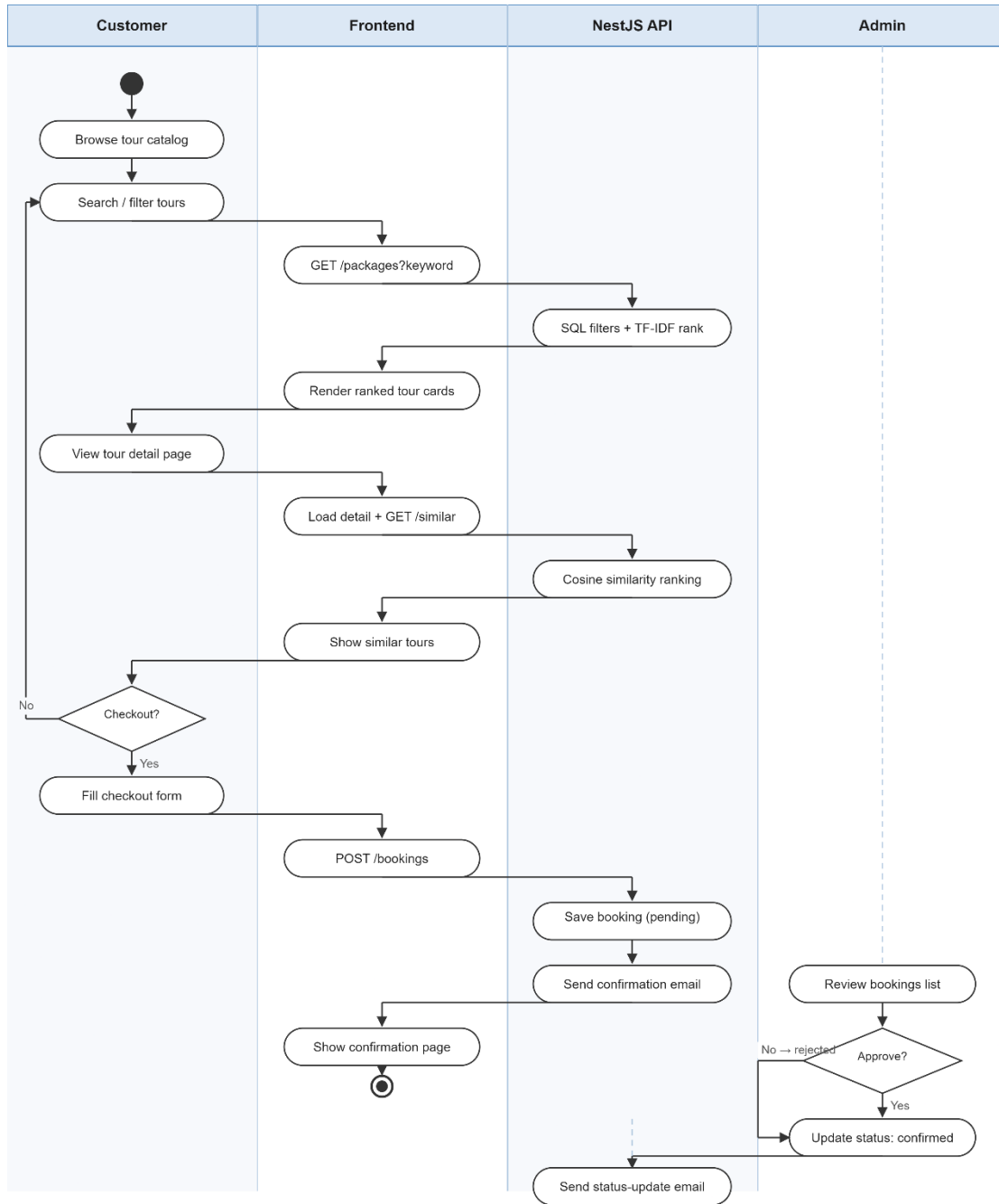


Figure 3.9: Activity Diagram of Tour Management System

Figure 3.9 illustrates the analysis activity diagram of the end-to-end booking process. It presents the high-level workflow across the Customer and Admin swimlanes, from tour discovery and Package selection through Booking submission, receipt upload, and admin

verification. The diagram focuses on the conceptual activities without implementation-specific details.

CHAPTER 4: SYSTEM DESIGN

System design is a crucial phase of the Software Development Life Cycle (SDLC) that transforms the system requirements identified during the analysis phase into a structured blueprint for software development. It defines the architecture, components, database structure, user interfaces, and communication mechanisms required to implement the proposed system efficiently. A well-designed system improves maintainability, scalability, performance, and overall software quality.

The Tour Management System follows a client-server, layered architecture, where the presentation, application, and database layers operate independently and communicate through RESTful APIs secured with JWT. This separation simplifies maintenance, enhances security, and allows each layer to be developed and updated independently.

- Frontend: Next.js 15 (App Router) with TypeScript, Mantine UI, and Tailwind CSS powers both the public catalogue and the protected admin portal.
- Backend: NestJS (Node.js/TypeScript) provides REST APIs (controllers for Packages, Bookings, Contacts, Auth, and Mail) for business logic, validation, authentication, and email dispatch.
- Algorithm Layer: Two TypeScript utility modules implement TF-IDF search ranking and Cosine Similarity content-based recommendations within the Package service.
- Database: MySQL, accessed through TypeORM entities and repositories, stores packages (and their related sub-entities), bookings, users, and contact messages as relational tables.
- File Storage: Multer stores uploaded receipt PDF files on the server's local filesystem under uploads/receipts/, with the relative path recorded in the booking record.
- Mail Service: @nestjs-modules/mailer with Nodemailer dispatches HTML transactional emails for booking confirmation and status updates.

- Security: JWT authentication (via @nestjs/jwt and passport-jwt) ensures only authorized admin accounts can access package management, booking management, and analytics endpoints.
- Deployment: Frontend on Vercel/Netlify, API on Render/Railway, and MySQL on a managed database host, all communicating over HTTPS/REST.

4.1 Design of the System

4.1.1 Refined Class Diagram

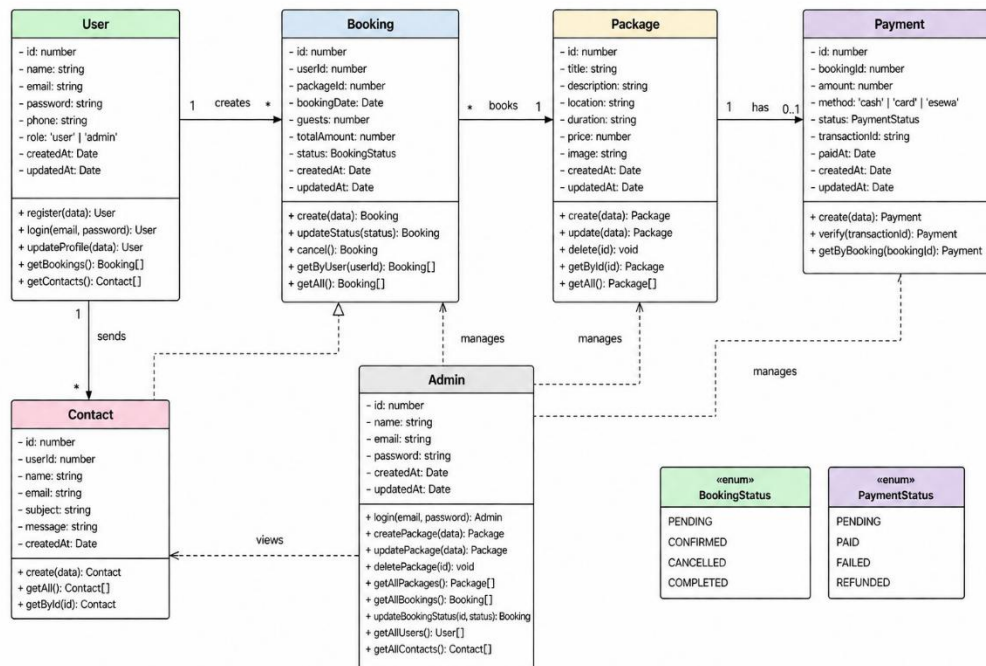


Figure 4.1: Refined Class Diagram of Tour Management System

4.1.2 Refined Object Diagram

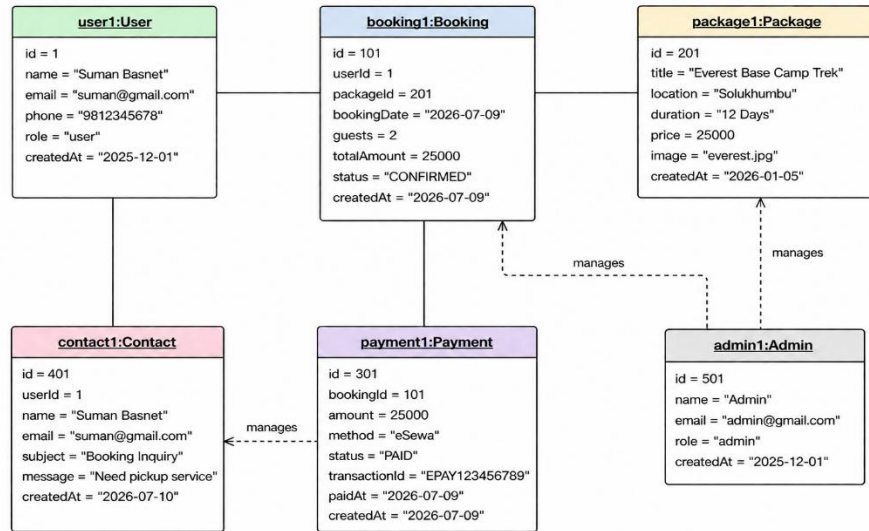


Figure 4.2: Refined Object Diagram of Tour Management System

4.1.3 Refined State Diagram

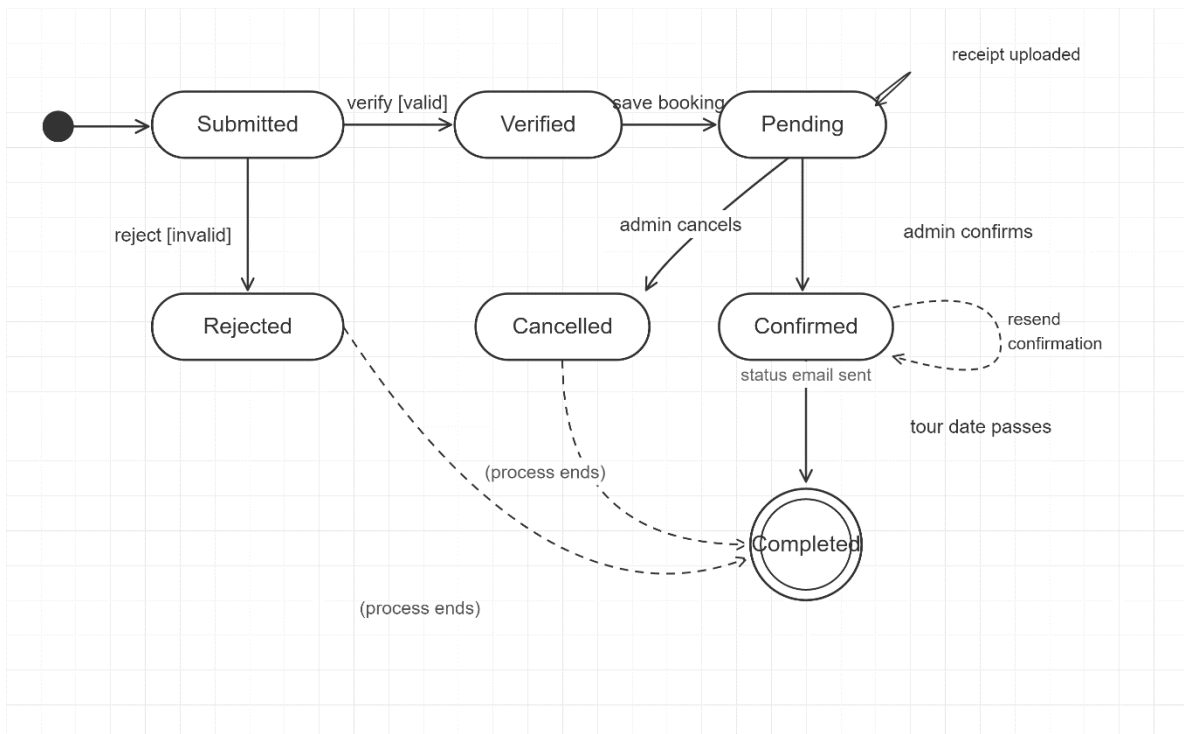


Figure 4.3: Refined State Diagram of Tour Management System

4.1.4 Refined Sequence Diagram

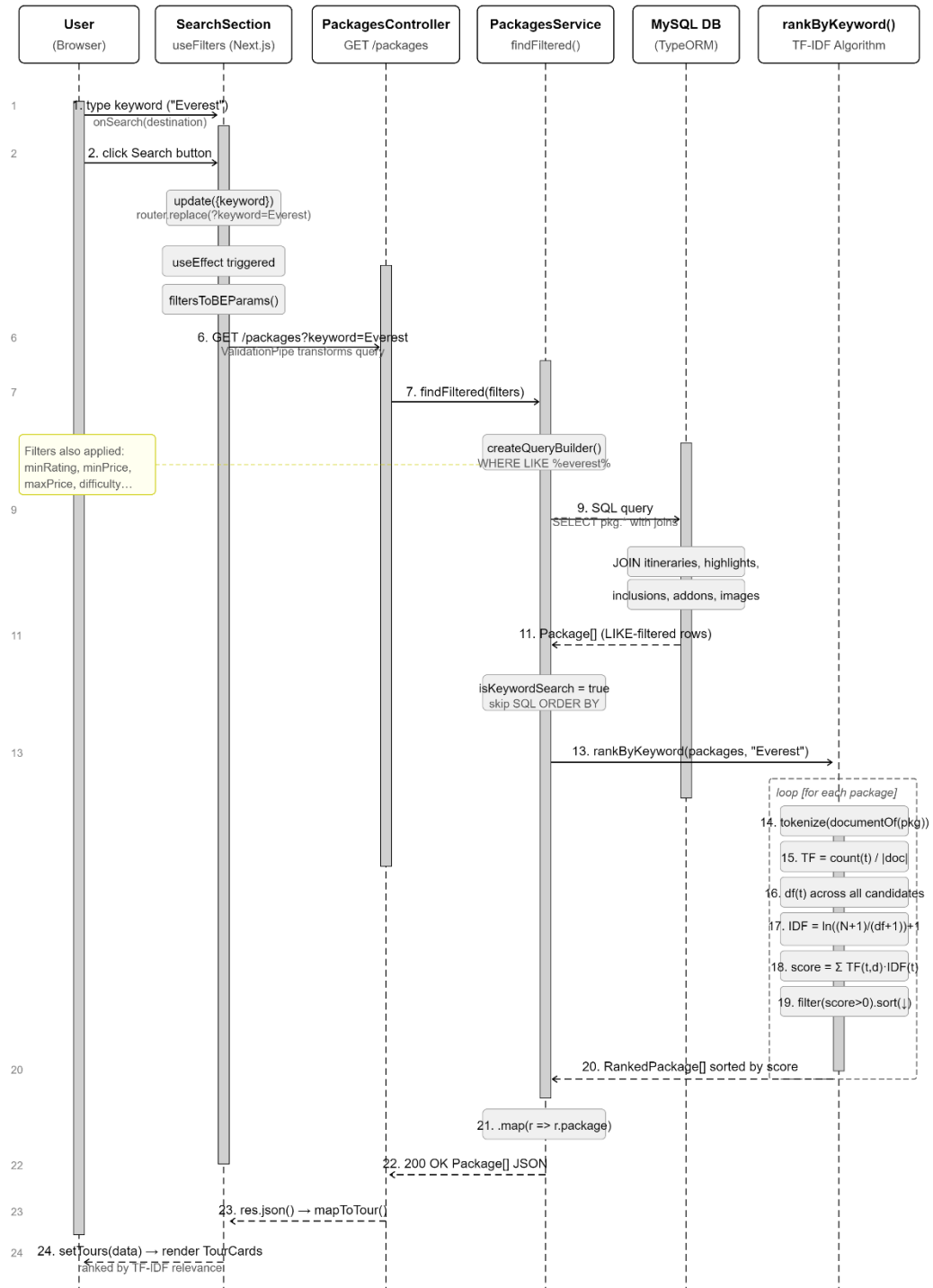


Figure 4.4: Sequence Diagram - Customer Searches of Tour Management System

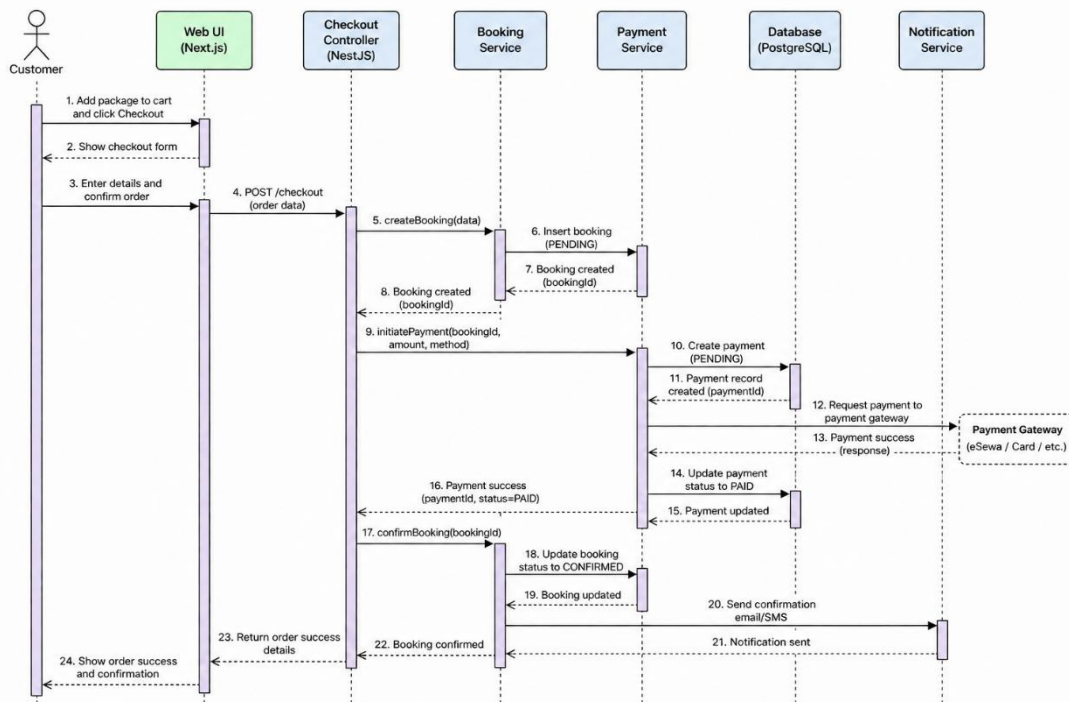


Figure 4.5: Sequence Diagram - Customer Checkout of Tour Management System

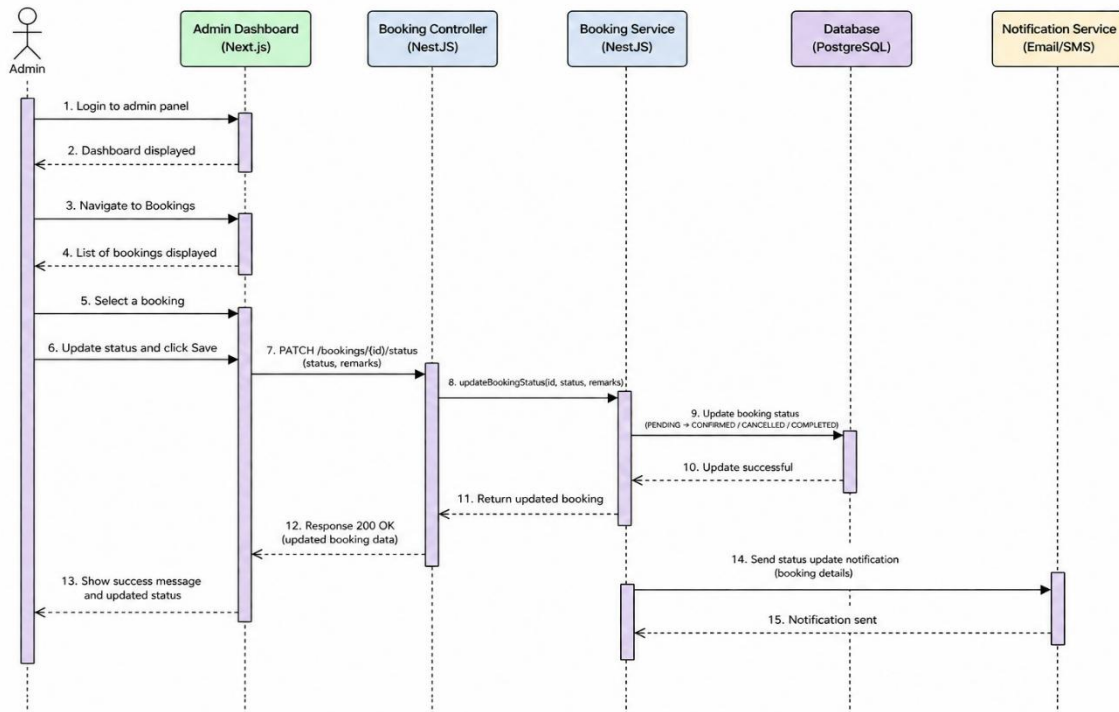


Figure 4.6: Sequence Diagram (Admin Updates Booking) of Tour Management System

4.1.5 Refined Activity Diagram

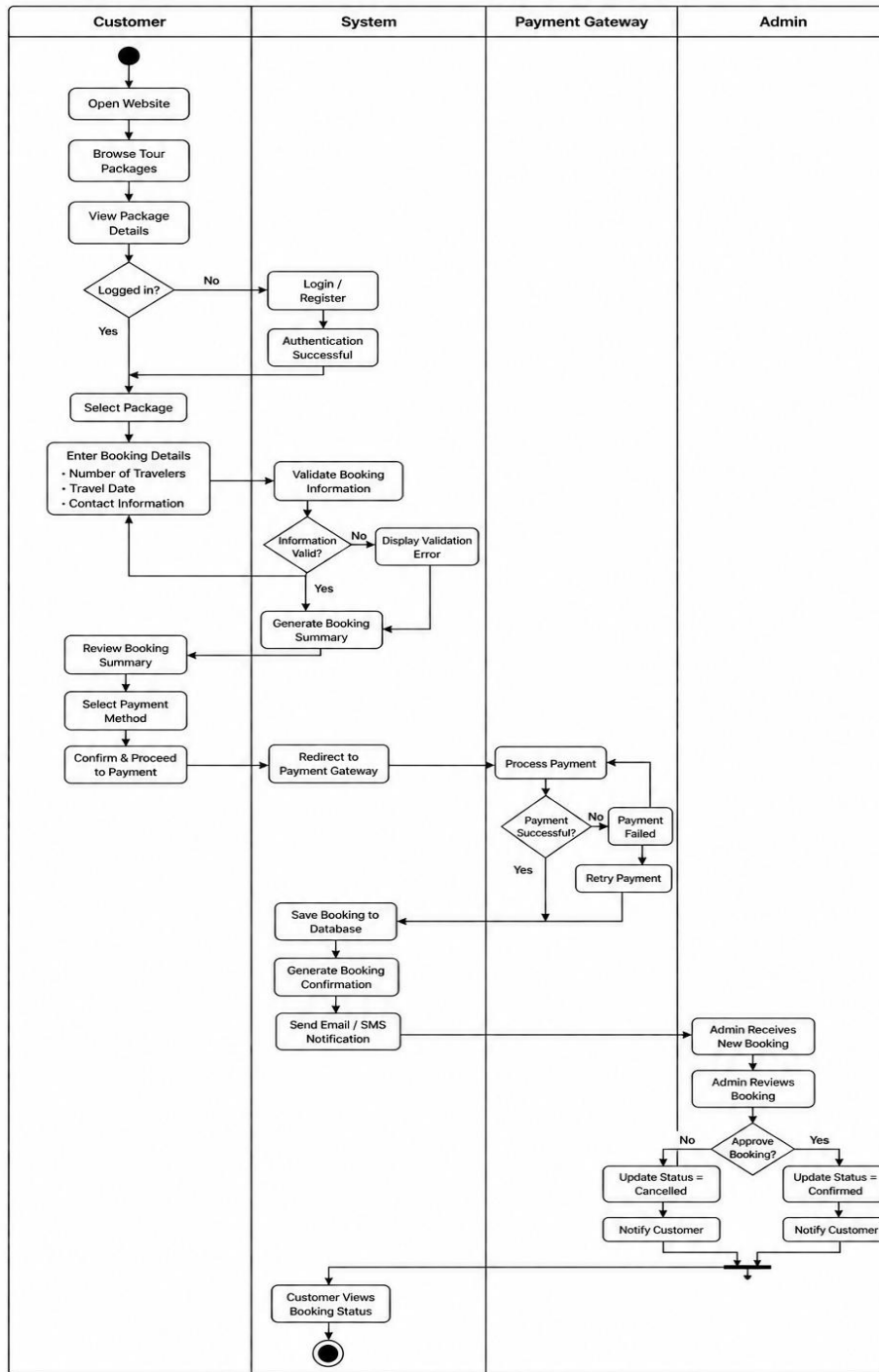


Figure 4.7: Refined Activity Diagram of Tour Management System

4.1.6 Components Diagram

The component diagram shows the internal software architecture by breaking the system into major functional units and illustrating how they communicate, independent of physical deployment. The Next.js Frontend talks to the NestJS API Gateway over REST/HTTPS, while the API's controllers depend on the AuthModule for JWT-based authentication and route guarding. The three domain modules PackagesModule (with the AlgorithmUtils for TF-IDF and Cosine Similarity), BookingsModule (with the MailModule for transactional email and Multer for receipt uploads), and ContactsModule each read and write against the shared MySQL database through TypeORM. The core idea is loose coupling: each module exposes a clear interface and hides its internal logic, so authentication, business logic, shared algorithms, file storage, email dispatch, and the database can each be built, updated, or scaled independently.

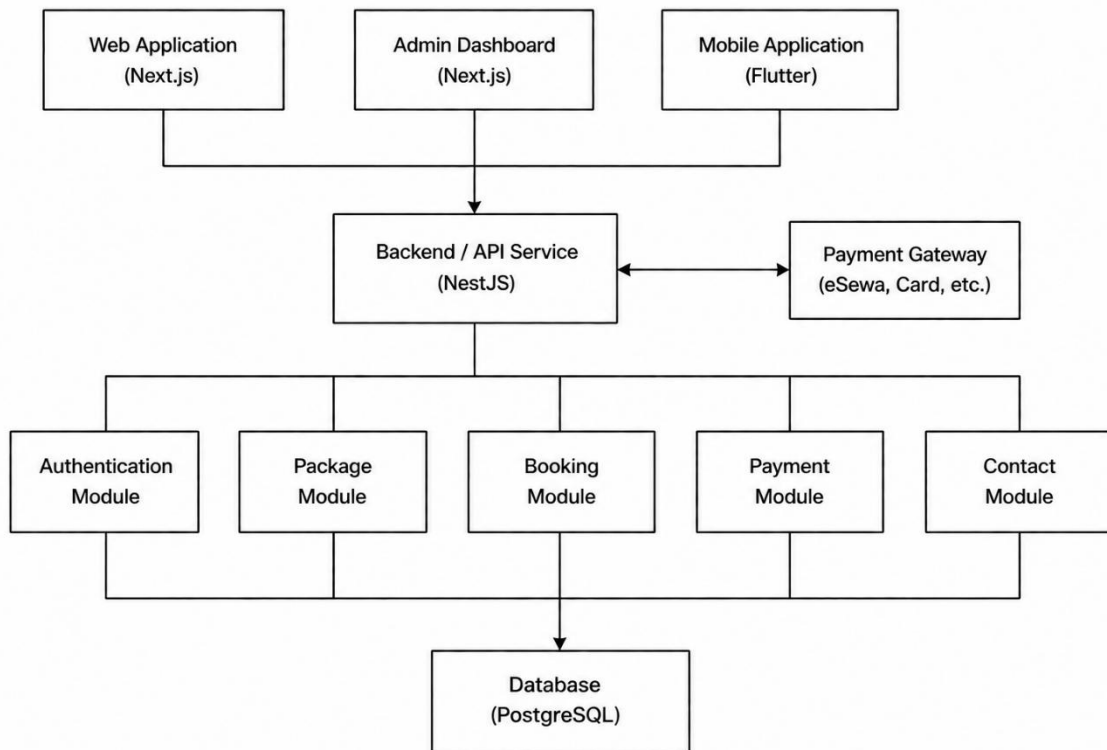


Figure 4.8: Component Diagram of Tour Management System

4.1.7 Deployment Diagram

The deployment diagram shows the physical infrastructure of the system the actual devices, servers, and hosting environments where each software component runs, along with the protocols used to communicate between them. In this design, the Client Browser (desktop/mobile) connects over HTTPS to the Frontend Hosting environment (Vercel/Netlify), which serves the built Next.js application. The frontend communicates with the API Hosting environment (Render/Railway), which runs the NestJS API on Node.js, over HTTPS/REST (JSON). The API host connects to a Managed MySQL instance (e.g. PlanetScale or Amazon RDS) over the MySQL wire protocol (TLS) for persistent data storage, and writes uploaded receipt PDFs to its local filesystem or a cloud storage bucket. The core idea is to map the logical components onto real-world, independently scalable hosting environments.

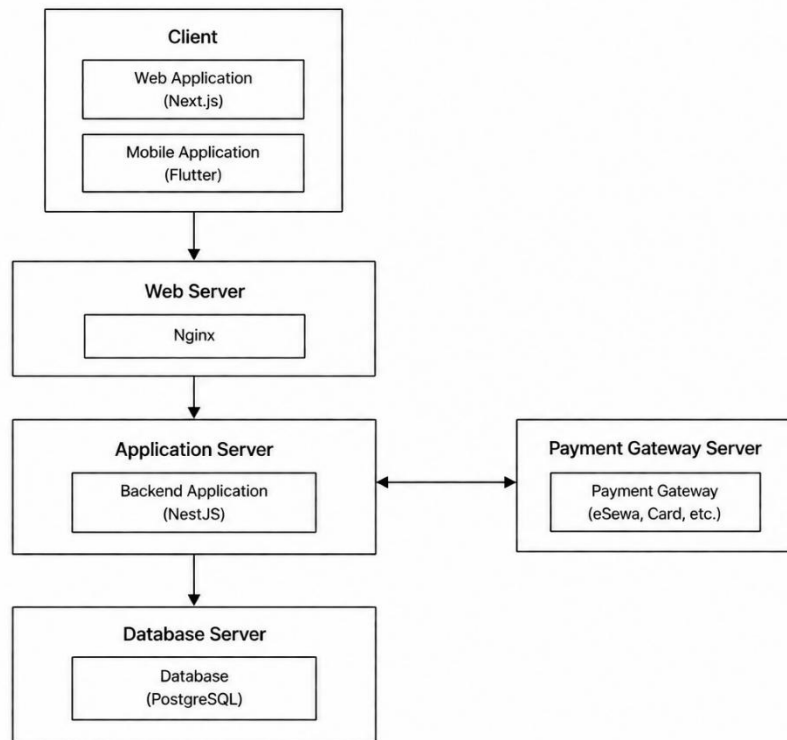


Figure 4.9: Deployment Diagram of Tour Management System

4.2 Algorithm Details

Two deterministic, hand-written algorithms sit behind the system's headline discovery features. Unlike a machine-learning model, each is a fixed, explainable procedure, so its output can always be traced back to a specific input and formula which matters in a commercial context where every search ranking and recommendation needs to be justifiable and reproducible.

4.2.1 TF-IDF Search Relevance Ranking (Information Retrieval Algorithm)

Before results are returned for a keyword query, the algorithm treats each active tour as a "document" constructed from its name, description, location, category, difficulty, and tagline, concatenated and tokenized (lower-cased, punctuation stripped, English stopwords removed). For each query term, Term Frequency (TF) is computed as the ratio of the term's occurrence count in a tour's document to the document's total token count. Inverse Document Frequency (IDF) is computed as $\ln((N+1)/(df+1))+1$, where N is the number of candidate tours and df is the number of tours whose document contains the term giving rare, distinctive terms (e.g. "Everest") a higher weight than common terms (e.g. "tour"). Each tour's relevance score is the sum of $TF \times IDF$ over all query terms; tours with a score of zero (no query term found) are excluded; remaining tours are returned sorted by descending score. This algorithm is $O(n \cdot L)$ in complexity, where n is the number of candidate tours and L is the average document length.

4.2.2 Content-Based Recommendation Engine (Cosine Similarity)

When a user views a tour detail page, the recommendation engine computes similarity scores between the viewed tour and every other active tour. Each tour is first converted into a numeric feature vector: numeric attributes (price, duration in days, rating) are min-max normalized to $[0,1]$; categorical attributes (category, difficulty, location) are one-hot encoded using the universe of all distinct values across the catalogue; boolean attributes (familyFriendly, freeCancellation) are included directly as 0/1 values. Similarity between two tours A and B is then computed as $\text{sim}(A,B) = (A \cdot B) / (\|A\| \times \|B\|)$, the cosine of the angle between their feature vectors, yielding a value in $[0,1]$ 1 meaning the tours are feature-identical and 0 meaning they share nothing in common. Because every feature is

normalized to $[0,1]$ before comparison, no single field (e.g. price) can dominate the similarity score. The top- N highest-scoring tours are returned. The algorithm is $O(n \cdot d)$ to build all feature vectors plus $O(n \log n)$ to sort, where d is the feature vector length trivial for a catalogue of this size. Because recommendations are based on tour attributes rather than booking history, the system avoids the cold-start problem: a brand-new tour with zero bookings can be recommended correctly from day one.

CHAPTER 5: IMPLEMENTATION AND TESTING

5.1 Implementation

With the requirements settled in Chapter 3 and the architecture worked out in Chapter 4, this chapter turns to how the Tour Management System was actually built. It covers the technologies used, how each functional module was implemented, and the testing carried out to confirm the system works as intended.

The system is built with Next.js (App Router, TypeScript) on the client, NestJS/TypeORM for the core backend, and MySQL as the relational database. Unlike a system with a separate machine-learning service, all analytical work here search ranking and tour recommendations is handled by the same two TypeScript algorithm utilities inside the NestJS Package service, so there is no second service to keep in sync; the whole system communicates over a single REST API.

Two roles drive the system Customer (public, unauthenticated) and Admin (JWT-authenticated) and every protected route checks a JWT issued at login to confirm identity before letting a request through. Customers browse packages, complete bookings, upload receipts, and receive emails; Admins manage packages, update booking statuses, download receipts, and monitor analytics. The recommendation engine and search ranking sit on top of the package data these interactions generate.

5.1.1 Tools Used

Table 5.1: Tools and Technologies Used in Tour Management System

Category	Tool / Technology	Role in the Project
Diagramming	draw.io	UML class/object/state/sequence/activity/component/deployment diagrams from Chapters 3–4

Category	Tool / Technology	Role in the Project
Frontend	Next.js 15 (App Router), TypeScript	Building the public tour catalogue and admin portal interfaces
UI Library	Mantine UI, Tailwind CSS	Tables, modals, forms, charts, and layout styling
Backend	NestJS (Node.js, TypeScript)	Core REST APIs (packages, bookings, contacts, auth, mail)
Data Layer	TypeORM	Entity modeling and repository-based queries against MySQL
Database	MySQL	Storing packages, bookings, users, contacts, and receipt paths
Validation	class-validator, class-transformer	DTO-level request validation before business logic runs
Security	JWT (@nestjs/jwt, passport-jwt)	Stateless admin authentication and route protection
Security	bcryptjs	Hashing stored admin passwords
File Upload	Multer	PDF receipt upload with MIME-type validation and size limits
Email	@nestjs-modules/mailer, Nodemailer	HTML transactional emails for booking confirmation and status updates
Networking	Axios	Frontend HTTP calls to the NestJS backend

Category	Tool / Technology	Role in the Project
State/UI (Admin)	Recharts, TanStack Table	Revenue chart and sortable booking/customer/tour tables in the admin dashboard
Algorithms	Custom TypeScript (AlgorithmUtils)	TF-IDF search ranking and Cosine Similarity recommendation
API Testing	Postman	Testing REST endpoints independently during development
Editor	VS Code	Writing and debugging the codebase
Version Control	Git / GitHub	Source management and collaboration
Package Management	npm	Dependency management for frontend and backend
Browser	Google Chrome	Manual frontend testing

5.1.2 Implementation Details of Modules

The system is organized around the object model introduced in Chapter 3 (Package, Booking, User, Contact, TourItinerary, TourHighlight, TourInclusion, TourAddon, TourImage, AlgorithmModule). Each module below maps onto one or more of these classes.

5.1.2.1 Authentication and Admin Access

The admin account is a row in the users table with role = "admin". Registration hashes the submitted password with bcrypt before it is stored; nothing plaintext ever touches the database. Logging in checks the submitted password against that hash and, on success, returns a signed JWT carrying the user's ID and role, through a single login endpoint. That token is what makes admin route protection work in practice: a NestJS JwtAuthGuard decodes and verifies it on every protected route (packages admin endpoints, bookings, dashboard) and rejects the request outright if no valid token is present. A Next.js middleware on the frontend reads the JWT from cookies and redirects unauthenticated visitors away from /dashboard and its sub-routes.

5.1.2.2 Package Management Module

This module implements the Package class and its associated sub-entities (TourItinerary, TourHighlight, TourInclusion, TourAddon, TourImage). Admin can create a package with all related data in a single POST request (cascade insert), update it with PUT, toggle its active status with PATCH /:id/toggle-active, and permanently delete it. The public GET /packages endpoint accepts filter parameters (keyword, category, difficulty, minPrice, maxPrice, minDays, maxDays, minRating, familyFriendly, freeCancellation) and applies the TF-IDF ranking algorithm when a keyword is supplied. The GET /packages/:id/similar endpoint invokes the Cosine Similarity algorithm and returns the top-N recommended tours. An auto-generated URL slug is produced from the package name on every insert and update via a TypeORM @BeforeInsert/@BeforeUpdate hook.

5.1.2.3 Booking Module

This module implements the Booking class. A customer submits a booking through POST /bookings with all traveler details, selected add-ons (stored as a JSON column), payment method, total amounts, and departure date. On creation, the BookingsService calls the MailService to dispatch an HTML booking-confirmation email. Admin can view all bookings (GET /bookings), update status (PATCH /bookings/:id/status which triggers a status-update email), and upload a receipt file (POST /bookings/:id/receipt). The receipt upload uses Multer with diskStorage, validates that only PDF files are accepted (up to 10 MB), saves the file under uploads/receipts/ with a unique timestamped filename, and

records the relative path in the booking's receiptPath column. Admins can also re-trigger the confirmation email via POST /bookings/:id/resend-confirmation.

5.1.2.4 Search Ranking Algorithm Module (TF-IDF)

This module implements the TF-IDF relevance ranking utility. When GET /packages is called with a keyword query parameter, the PackagesService first fetches all matching active tours from MySQL (using a LIKE filter as a pre-filter), then passes the candidate list and the keyword string to rankByTfIdf(). Each tour is converted into a document by concatenating its name, description, location, category, difficulty, and tagline, then tokenized (lowercased, punctuation stripped, English stopwords removed). TF is computed per term per document; IDF is computed across the candidate set. Each tour receives a score = $\Sigma(TF \times IDF)$ over all query terms; tours with score 0 are dropped; the remainder are sorted by descending score and returned. The result is that a tour with "Everest" in its name and description ranks substantially higher than one that merely mentions it in passing.

5.1.2.5 Recommendation Algorithm Module (Cosine Similarity)

This module implements the Cosine Similarity content-based recommendation engine. When GET /packages/:id/similar is called, the PackagesService fetches all active tours and passes them along with the target tour's ID and the desired limit to recommendByCosine(). All tours are first converted to feature vectors: price, days, and rating are min-max normalized over the full catalogue; category, difficulty, and location are one-hot encoded; familyFriendly and freeCancellation are included as binary values. The dot product and Euclidean norms are computed between the target tour's vector and every other tour's vector; similarity scores are computed as the cosine of the angle. Tours are sorted by descending similarity and the top-N (excluding the target itself) are returned. These results power the "You Might Also Like" section on every tour detail page.

5.1.2.6 Mail Service Module

This module implements the MailService class. It dispatches two types of transactional email: booking confirmation (sent automatically when a booking is created) and status update (sent when the admin changes a booking's status to confirmed, cancelled, or pending). Both email types are formatted as HTML using a shared shell function that

renders a branded header, a highlighted booking-details info box with a coloured status badge, and a footer. The service catches email delivery failures and logs them without propagating errors to the caller, ensuring that a mail delivery problem never crashes the booking creation flow.

5.1.2.7 Contact Module

This module implements the Contact class. A visitor submits a contact inquiry through POST /contact with their name, email, subject, and message. The inquiry is stored in the contacts table with a default read = false flag. The admin can view all messages from the admin dashboard Messages page and mark individual messages as read. This provides a simple, centralized inbox for customer inquiries submitted through the public contact form.

5.1.2.8 Admin Dashboard Module

The admin dashboard is a protected Next.js route tree (/dashboard) guarded by the JWT middleware. The main overview page renders a StatsGrid (live revenue, total bookings, active tours, and pending bookings derived from the bookings and packages APIs), a RevenueChart (monthly revenue trend using Recharts), a PopularTours table, a BookingTable showing recent bookings with status badges, and a CustomerTable. Sub-pages (/dashboard/tours, /dashboard/bookings, /dashboard/customers, /dashboard/messages, /dashboard/reviews, /dashboard/settings) each fetch their data from the corresponding backend endpoints and render it using TanStack Table for sortable, filterable grids. The AddTourModal allows admins to create or edit packages directly from the tours sub-page without navigating away.

5.2 Testing

Testing covered the system at two levels: individual modules and algorithms in isolation (unit testing), and complete user workflows spanning the frontend, backend, and database (system testing).

5.2.1 Unit Testing

Table 5.2: Unit Test Cases of Tour Management System

ID	Module	What Was Tested	Expected Outcome	Result
U-01	Authentication	Register a new admin account	Account created, password stored hashed	Pass
U-02	Authentication	Log in with correct credentials	JWT issued, login succeeds	Pass
U-03	Authentication	Log in with wrong password	Login rejected with 401	Pass
U-04	Authentication	Access admin route without JWT	Request blocked with 401	Pass
U-05	Package Module	Create a package with itinerary and highlights	Package and sub-entities saved (cascade insert)	Pass
U-06	Package Module	Toggle a package to inactive	isActive set to false; excluded from public GET /packages	Pass
U-07	Package Module	Filter packages by category and difficulty	Only matching active packages returned	Pass
U-08	TF-IDF Algorithm	Search "Everest" against catalogue	Everest Base Camp Tour ranked above generic trekking tours	Pass

ID	Module	What Was Tested	Expected Outcome	Result
U-09	TF-IDF Algorithm	Search with common term "tour"	Results ordered by relative relevance, not all equally scored	Pass
U-10	Cosine Similarity	Get similar tours for a high-altitude trek	Top results share category, difficulty, and price range	Pass
U-11	Cosine Similarity	Get similar tours for a cultural heritage package	Top results are culture-category packages, not adventure treks	Pass
U-12	Booking Module	Create a booking with add-ons	Booking saved; addons Total and total Amount correct; confirmation email dispatched	Pass
U-13	Booking Module	Upload a PDF receipt to a booking	File saved to uploads/receipts/; receipt Path updated in booking record	Pass
U-14	Booking Module	Upload a non-PDF file as receipt	Request rejected with 400 "Only PDF files are accepted"	Pass

ID	Module	What Was Tested	Expected Outcome	Result
U-15	Booking Module	Update booking status to confirmed	Status updated; status-update email dispatched to customer	Pass
U-16	Booking Module	Resend confirmation email	Confirmation email re-dispatched for the booking	Pass
U-17	Contact Module	Submit a contact form inquiry	Contact record saved with read = false	Pass
U-18	Mail Service	Email delivery failure during booking creation	Error logged; booking creation still completes successfully	Pass
U-19	Package Module	Delete a package	Package and all sub-entities removed from database (cascade)	Pass

5.2.2 System Testing

Table 5.3: System Test Cases of Tour Management System

ID	Scenario	Steps	Expected Outcome	Result
S-01	End-to-end tour discovery	Customer opens catalogue, applies filters for category=trek and difficulty=moderate, searches "Annapurna"	Filtered results ranked by TF-IDF; "Annapurna Circuit" ranked first	Pass
S-02	Recommendation on detail page	Customer views "Everest Base Camp Trek" detail page	"You Might Also Like" shows similar high-altitude trekking packages, not cultural tours	Pass
S-03	Full checkout flow (Khalti)	Customer selects tour, fills personal details, picks add-ons, selects Khalti, uploads PDF receipt, submits	Booking created with status=pending, receipt stored; confirmation email received	Pass
S-04	Full checkout flow (Card)	Customer selects tour, fills details, selects Card payment, submits without receipt upload	Booking created with status=pending; confirmation email received	Pass

ID	Scenario	Steps	Expected Outcome	Result
S-05	Admin confirms booking	Admin changes booking status to confirmed from dashboard	Status updated in DB; customer receives status-update confirmation email	Pass
S-06	Admin cancels booking	Admin changes booking status to cancelled	Status updated; customer receives cancellation email	Pass
S-07	Admin creates new package	Admin uses AddTourModal to create a package with 3 itinerary days, 4 highlights, 5 inclusions, and 2 add-ons	Package and all sub-entities saved; appears on public catalogue	Pass
S-08	Admin deactivates package	Admin toggles a package to inactive	Package disappears from public catalogue; still visible in admin tour list	Pass

ID	Scenario	Steps	Expected Outcome	Result
S-09	Admin views analytics	Admin opens dashboard overview page	StatsGrid shows live revenue, booking count, active tours, and pending count; RevenueChart renders monthly bars	Pass
S-10	Admin reads contact message	Admin opens Messages page and marks an inquiry as read	Contact record updated: read = true; message no longer highlighted as unread	Pass
S-11	Unauthenticated admin access	Unauthenticated user navigates directly to /dashboard/bookings	Redirected to admin login page by Next.js middleware	Pass
S-12	Database persistence	Application restarted after packages and bookings were created	All records remain intact in MySQL	Pass

5.3 Result Analysis

Testing across both levels did not surface anything that broke the system's core workflows. The JWT-based authentication model held up consistently requests without a valid token

are rejected outright at the guard level, regardless of which admin endpoint is targeted and the booking lifecycle (pending → confirmed/cancelled) moved correctly through every state, with the appropriate transactional email dispatched at each transition.

The two core algorithms behaved exactly as specified, which is expected given they are deterministic procedures rather than statistical models. The TF-IDF ranking correctly elevated tours with concentrated keyword matches in their name and description above those with sparse or incidental matches; the Cosine Similarity engine correctly clustered tours by shared attribute profiles cultural packages were recommended alongside cultural packages, challenging high-altitude treks alongside other challenging treks and did so correctly for packages that had zero bookings, confirming the cold-start advantage. Because neither algorithm relies on statistical inference, there is no accuracy percentage to report: each either matches its specification exactly or represents a defect, and no defects were found during this round of testing.

The receipt upload flow performed correctly across all test cases: PDF files up to 10 MB were accepted and stored, non-PDF uploads were rejected with clear error messages, and the stored path was correctly returned when the admin viewed a booking. Integration between the Next.js frontend and the NestJS backend did not cause friction every request went through the single REST API, and the Next.js middleware correctly protected every admin route without requiring separate authentication logic per page.

CHAPTER 6: CONCLUSION AND FUTURE RECOMMENDATIONS

6.1 Conclusion

Sajilo Yatra- A Tour Management System, successfully digitizes the core workflows of a Nepal-based tour operator: public package discovery with intelligent filtering, keyword search ranked by TF-IDF relevance, content-based tour recommendations powered by Cosine Similarity, multi-step online booking with optional add-ons, local-payment receipt handling, automated transactional email communication, and a comprehensive admin dashboard for package management, booking management, and business analytics.

The system addresses the key limitations of manual and offline tour-booking processes: customers can now discover and book packages at any time through a responsive web interface without contacting the operator directly; the TF-IDF engine ensures that search results reflect genuine relevance rather than an arbitrary default sort; and the Cosine Similarity recommender helps customers discover related tours they might not have searched for, increasing potential conversions. On the admin side, package management, booking status updates, receipt review, and analytics are consolidated in a single, role-protected dashboard, replacing scattered spreadsheets and manual follow-up.

The project demonstrates that non-trivial, explainable algorithmic logic information retrieval (TF-IDF) and content-based filtering (Cosine Similarity) can be integrated cleanly into a production-grade NestJS REST API in plain TypeScript without any external machine-learning infrastructure, making the system lightweight, maintainable, and fully traceable. The JWT-secured backend and Next.js middleware route protection ensure that admin functionality remains inaccessible to unauthenticated users, while the modular NestJS architecture (one module per domain) keeps the codebase organized and extensible.

In conclusion, the Tour Management System provides a secure, modern, and scalable platform that successfully integrates package discovery, intelligent search and recommendations, booking management, local payment handling, and automated

communication into a single cohesive system tailored to the needs of Nepal's tour and trekking industry.

6.2 Future Recommendation

While the Tour Management System meets its defined objectives, several enhancements could be made in future iterations to extend its capabilities:

- **Real Payment Gateway Integration:** Integrating the official Khalti and eSewa payment APIs, and a card-payment provider such as Stripe, would allow fully automated payment verification and eliminate the need for manual receipt review by the admin.
- **Collaborative Filtering Recommendations:** Combining the existing content-based Cosine Similarity engine with collaborative filtering (based on actual booking history and user behaviour) would produce a hybrid recommendation system that improves in accuracy as the platform accumulates more bookings.
- **Customer Accounts and Review System:** Allowing customers to create accounts, track their booking history, submit star ratings, and write reviews would add social proof to package listings and provide richer data for collaborative filtering.
- **Mobile Application:** Developing a React Native (Expo) or Flutter mobile application consuming the same REST API would extend the platform to mobile users and enable push notifications for booking status changes.
- **Multi-Language Support (i18n):** Adding Nepali-language support to the public catalogue and checkout flow would broaden accessibility for domestic travelers and improve conversion among local customers.
- **Advanced Analytics and Reporting:** Extending the admin dashboard with exportable CSV/PDF reports, revenue forecasting, and cohort analysis (repeat customers, peak booking periods) would give the operator deeper business intelligence.

- Automated Availability and Capacity Management: Tracking the maximum number of travelers per departure date and automatically closing bookings when capacity is reached would prevent overbooking without manual monitoring.
- Multi-Operator / Multi-Tenant Support: Extending the architecture to support multiple independent tour operators under a single platform instance would enable the system to scale into a marketplace model.

REFERENCES

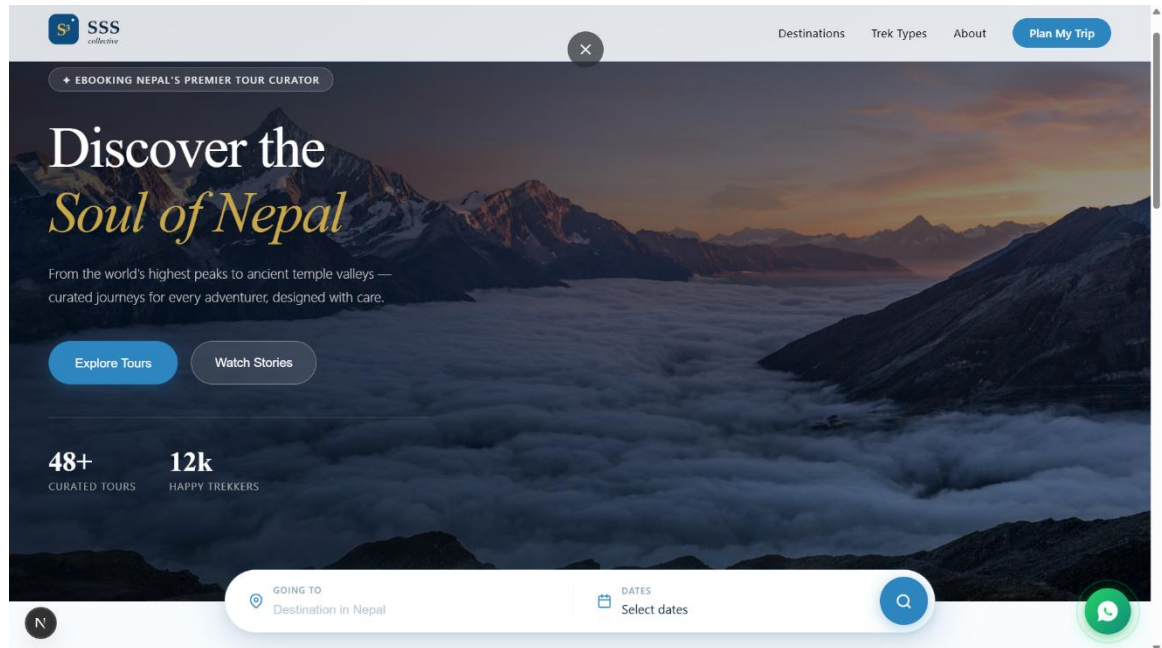
- [1] M. J. Abu-AlShaer, H. A. Hasan, S. I. Mustafa, D. Khlaponin, and K. Krasovska, "A Comprehensive Online Tourism Management System Revolutionizes Travel," Proceedings of the 35th FRUCT Conference, ISSN 2305-7254.
- [2] Q. Odeniran, H. Wimmer, and C. Rebman, "Node.js or PHP? Determining the Better Website Server Backend Scripting," Issues in Information Systems, 2023.
- [3] J. E. Ramos, "Using TF-IDF to Determine Word Relevance in Document Queries," Proceedings of the First Instructional Conference on Machine Learning, 2003.
- [4] B. Sarwar, G. Karypis, J. Konstan, and J. Riedl, "Analysis of Recommendation Algorithms for E-Commerce," GroupLens Research Group, University of Minnesota.
- [5] N. B. Munkholm, R. A. Alphinias, and T. Tambo, "Collaborative Filtering, K-Nearest Neighbor and Cosine Similarity in Home Décor Recommender Systems – A Case Study," arXiv:2402.06233, 2024.
- [6] S. Dalimunthe, E. Hasri Putra, and M. A. Fadhly Ridha, "Restful API Security Using JSON Web Token (JWT) With HMAC-Sha512 Algorithm in Session Management," IT Journal Research and Development, vol. 8, no. 1, pp. 81–94, 2023.
- [7] P. Acharya, "Adoption and Impact of Mobile Wallets and Digital Banking in Nepal," BMC Research Journal, vol. 5, no. 1, pp. 148–162, 2026.
- [8] NestJS, NestJS Documentation. Available: <https://docs.nestjs.com/>
- [9] Vercel, Next.js Documentation. Available: <https://nextjs.org/docs>

- [10] TypeORM, TypeORM Documentation. Available: <https://typeorm.io/>
- [11] Khalti, Khalti Payment Gateway Developer Documentation. Available: <https://docs.khalti.com/>
- [12] eSewa, eSewa Merchant Intesgration Documentation. Available: <https://developer.esewa.com.np/>
- [13] Tribhuvan University, Institute of Science and Technology, BIT404 Project Work Course Syllabus.

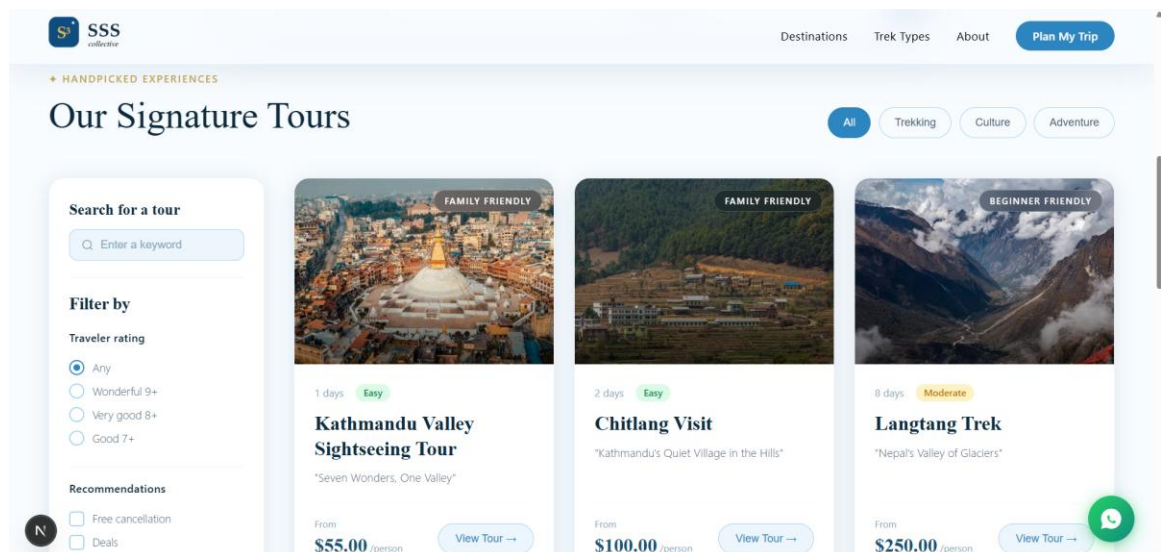
APPENDICES

APPENDIX A: SCREENSHOTS

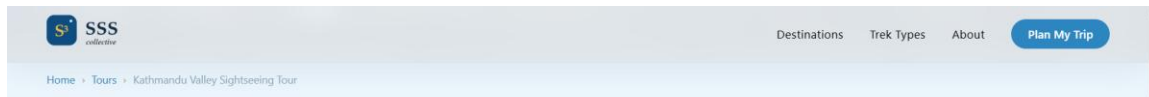
Landing Pages



- Destinations



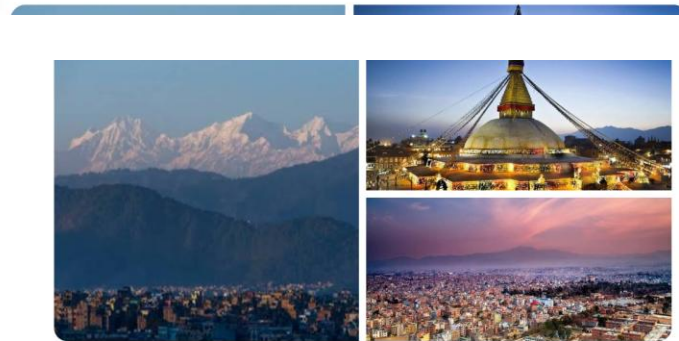
- Destinations Details Page



About this Tour

Kathmandu Valley holds seven UNESCO World Heritage monument zones packed into a single metropolitan area — an unusually dense concentration of living cultural heritage that most cities simply don't have. The tour typically covers the three historic Durbar Squares (Kathmandu, Patan, and Bhaktapur), each a former royal palace complex showcasing centuries of Newari wood and stone carving, plus the great Buddhist stupas of Swayambhunath and Boudhanath, the sacred Hindu temple of Pashupatinath, and the ancient hilltop shrine of Changu Narayan — believed to be the oldest Hindu temple in the valley. Unlike a trek, this is entirely accessible sightseeing: no altitude, no walking poles, comfortable for any fitness level or age group, and easily combined with shopping, food tours, or a half-day at leisure. It's commonly booked either as a standalone city experience for travelers with limited time, or bolted onto the front or back end of a trek itinerary (Langtang, EBC, etc.) to fill acclimatization or arrival/departure days productively rather than leaving them idle in a hotel. Because these are active religious and civic sites — not museum recreations — the experience shifts throughout the day: morning is quieter and better for photography at Swayambhunath and Boudhanath, while Pashupatinath is most atmospheric during evening aarti rituals along the Bagmati River.

Photo Gallery



Day-by-Day Itinerary

D1 Morning: Swayambhunath (quieter light, fewer crowds) — Kathmandu Durbar Square

\$55 / person

Base price per person. Customize your experience with add-ons below.

OPTIONAL ADD-ONS

- ☐ **Pashupatinath Evening Aarti Ritual** +\$15
Guided return visit at dusk for the sacred fire ritual along the Bagmati River
- ☐ **Bhaktapur Pottery Workshop** +\$20
Hands-on pottery-wheel session in Bhaktapur's famous Pottery Square
- ☐ **Thamel Food & Street Food Tour** +\$30
Evening guided food walk through Thamel — momos, Newari cuisine, local sweets
- ☐ **Private Photographer** +\$40
Dedicated photographer for the day, edited digital photo set delivered post-tour

OPTIONAL ADD-ONS

- ☐ **Pashupatinath Evening Aarti Ritual** +\$15
Guided return visit at dusk for the sacred fire ritual along the Bagmati River
- ☐ **Bhaktapur Pottery Workshop** +\$20
Hands-on pottery-wheel session in Bhaktapur's famous Pottery Square
- ☐ **Thamel Food & Street Food Tour** +\$30
Evening guided food walk through Thamel — momos, Newari cuisine, local sweets
- ☐ **Private Photographer** +\$40
Dedicated photographer for the day, edited digital photo set delivered post-tour

Total estimate **\$55**

Day-by-Day Itinerary

D1

Morning: Swayambhunath (quieter light, fewer crowds) → Kathmandu Durbar Square

D2

Midday: Lunch in Kathmandu/Thamel

D3

Afternoon: Boudhanath Stupa → Pashupatinath Temple (evening aarti ritual optional)

Tour Highlights

Kathmandu Durbar Square
Hanuman Dhoka Palace, Kumari Ghar (home of the Living Goddess), Taleju Temple

Swayambhunath ("Monkey Temple")
hilltop stupa with panoramic valley views, one of the oldest sites in the valley

Boudhanath Stupa
one of the largest stupas in the world, spiritual center of Tibetan Buddhism in Nepal

Pashupatinath Temple
Nepal's most sacred Hindu temple complex, cremation ghats along the Bagmati River

Booking Process

- Applying Add-ons

SSS
collective

Destinations
Trek Types
About
Plan My Trip

D3

Afternoon: Boudhanath Stupa → Pashupatinath Temple (evening aarti ritual optional)

Tour Highlights

Kathmandu Durbar Square
Hanuman Dhoka Palace, Kumari Ghar (home of the Living Goddess), Taleju Temple

Swayambhunath ("Monkey Temple")
hilltop stupa with panoramic valley views, one of the oldest sites in the valley

Boudhanath Stupa
one of the largest stupas in the world, spiritual center of Tibetan Buddhism in Nepal

Pashupatinath Temple
Nepal's most sacred Hindu temple complex, cremation ghats along the Bagmati River

What's Included

INCLUDED

- Private/shared vehicle transport between sites
- English-speaking licensed guide
- All entrance fees to listed sites

NOT INCLUDED

- Meals (unless specified)
- Personal shopping/tips
- Optional inner-temple donations at Pashupatinath

OPTIONAL ADD-ONS

☒
Pashupatinath Evening Aarti Ritual +\$15
Guided return visit at dusk for the sacred fire ritual along the Bagmati River

☒
Bhaktapur Pottery Workshop +\$20
Hands-on pottery-wheel session in Bhaktapur's famous Pottery Square

☒
Thamel Food & Street Food Tour +\$30
Evening guided food walk through Thamel — momos, Newari cuisine, local sweets

☒
Private Photographer +\$40
Dedicated photographer for the day, edited digital photo set delivered post-tour

Total estimate
\$160

Book This Tour

✓ Free cancellation up to 14 days before departure

Booking Flow

- Booking Page

[Destinations](#)
[Trek Types](#)
[About](#)
[Plan My Trip](#)

1

2

3

Your Details

Tell us who's joining this adventure.

FIRST NAME

LAST NAME

Shovit

Regmi

EMAIL ADDRESS

PHONE NUMBER

regmishovit12@gn

+9779866599038

COUNTRY OF RESIDENCE

NUMBER OF TRAVELERS

Nepal

2

SPECIAL REQUESTS (optional)

Dietary needs, accessibility requirements, etc.

Continue to Review →

Kathmandu Valley Sightseeing Tour

1 Days

Base price	\$55
× 2 travelers	\$110
Pashupatinath Evening Aarti Ritual	+\$15
Bhaktapur Pottery Workshop	+\$20
Thamel Food & Street Food Tour	+\$30
Private Photographer	+\$40
Total	\$320

Free cancellation up to 14 days before departure

- Booking Review

Review Your Booking

Please check all details before proceeding.

Tour

Kathmandu Valley Sightseeing Tour

Duration

1 Days

Difficulty

EASY

Name

Shovit Regmi

Email

regmishovit12@gmail.com

Phone

+9779866599038

Country

Nepal

Travelers

2

Add-ons

Pashupatinath Evening Aarti Ritual, Bhaktapur Pottery Workshop, Thamel Food & Street Food Tour, Private Photographer

☒ I agree to the [Terms of Service](#) and [Cancellation Policy](#).

← Back

Proceed to Payment! →

Kathmandu Valley Sightseeing Tour

1 Days

Base price	\$55
× 2 travelers	\$110
Pashupatinath Evening Aarti Ritual	+\$15
Bhaktapur Pottery Workshop	+\$20
Thamel Food & Street Food Tour	+\$30
Private Photographer	+\$40
Total	\$320

Free cancellation up to 14 days before departure

- Payment Page (Card + Khalti + eSewa)

✓

✓

3

Payment

Complete your booking securely.

Khalti

eSewa

Card

CARD NUMBER

1234 5678 9012 3456

EXPIRY

MM / YY

CVV

← Back

Pay & Confirm Book

Payments are encrypted and secure

EASY

Kathmandu Valley Sightseeing Tour

1 Days

Base price

\$55

x 2 travelers

\$110

Pashupatinath Evening Aarti Ritual

+\$15

Bhaktapur Pottery Workshop

+\$20

Thamel Food & Street Food Tour

+\$30

Private Photographer

+\$40

Total

\$320

Free cancellation up to 14 days before departure

✓

✓

3

Payment

Complete your booking securely.

Khalti

eSewa

Card

SCAN TO PAY WITH ESEWA

Send exactly **NPR 42,560** (i.e. \$320) to the eSewa QR above.

UPLOAD TRANSACTION RECEIPT *

Click to upload your receipt

PDF only - Max 10 MB

After completing payment in eSewa, download your receipt and upload it here. We will verify and confirm your booking within 2 business hours.

← Back

Submit & Confirm !

Payments are encrypted and secure

EASY

Kathmandu Valley Sightseeing Tour

1 Days

Base price

\$55

x 2 travelers

\$110

Pashupatinath Evening Aarti Ritual

+\$15

Bhaktapur Pottery Workshop

+\$20

Thamel Food & Street Food Tour

+\$30

Private Photographer

+\$40

Total

\$320

Free cancellation up to 14 days before departure

64

- Confirm Booking (User Side)

Confirm Your Booking

Please review the details below before we confirm.

Tour

Kathmandu Valley Sightseeing Tour

Duration

1 Days

Traveler

Shovit Regmi

Email

regmishovit12@gmail.com

Travelers

2 people

Payment

eSewa

Add-ons

Pashupatinath Evening Aarti Ritual,
Bhaktapur Pottery Workshop,
Thamel Food & Street Food Tour,
Private Photographer

Total due

\$320

Free cancellation up to 14 days before departure

Go Back

Yes, Confirm Booking

Booking Confirmed!

Your adventure is locked in. A confirmation has been sent to regmishovit12@gmail.com.

BOOKING REFERENCE

SH - X5B0IG

Kathmandu Valley Sightseeing Tour

1 days

CONFIRMED

Departure

Saturday, August 22, 2026

Travelers

2 people

Pay via

eSewa

Total Paid

\$320

WHAT HAPPENS NEXT?

A detailed itinerary and packing guide will be emailed within 24 hours.

Your trek coordinator will call you 7 days before departure.

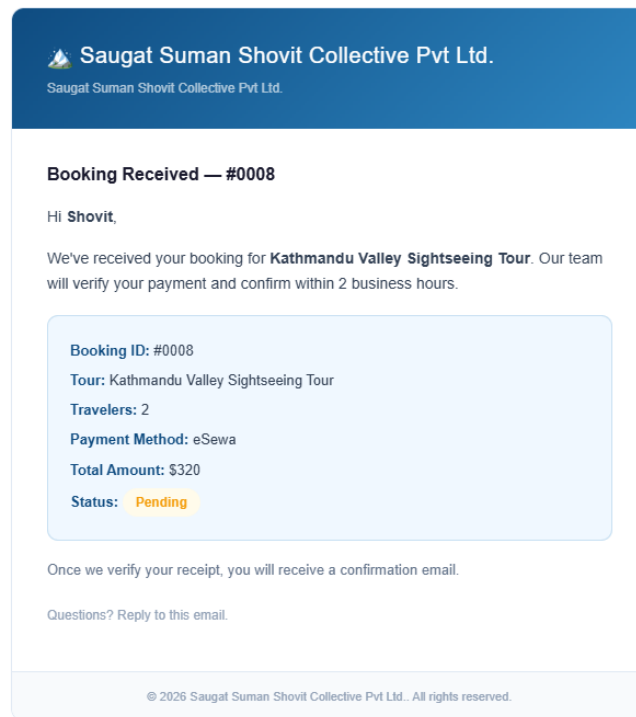
Permits and accommodation are arranged by our team.

Download Confirmation PDF

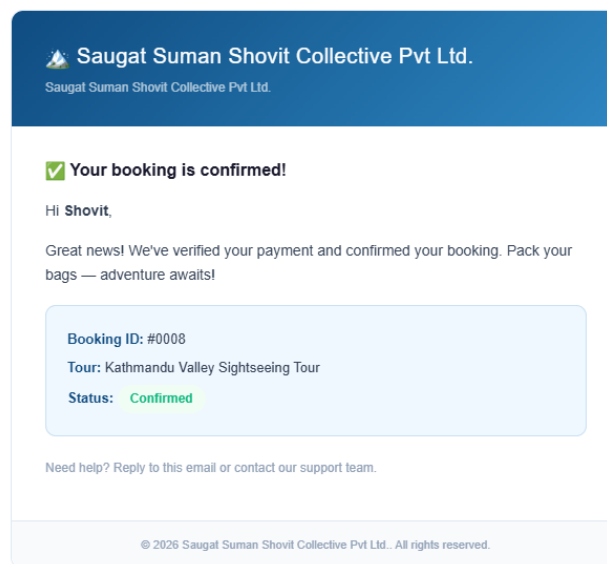
Resend Email

Browse More Tours

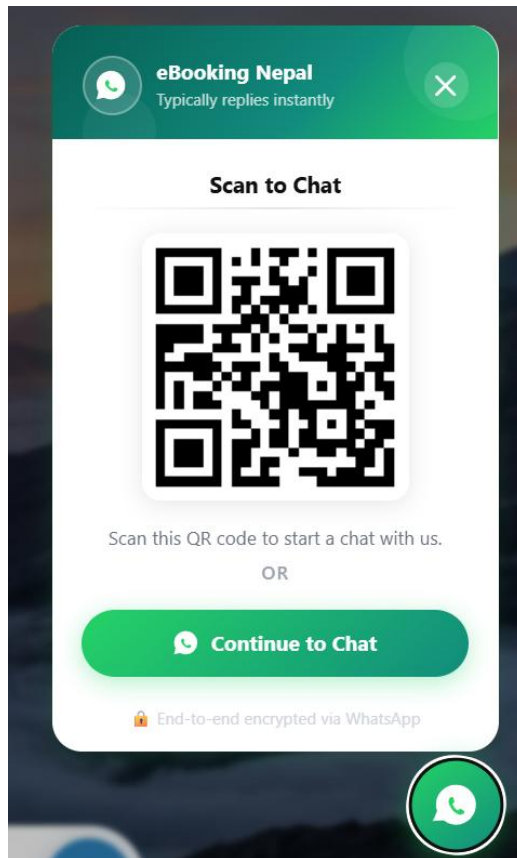
- Confirmation Mail (before admin confirmation)



- Confirmation Mail (after admin confirmation)



- Whatsapp Integration



Admin Pages

- Admin Login

ADMIN PANEL

Nepal Treks

Sign in to your admin account

EMAIL

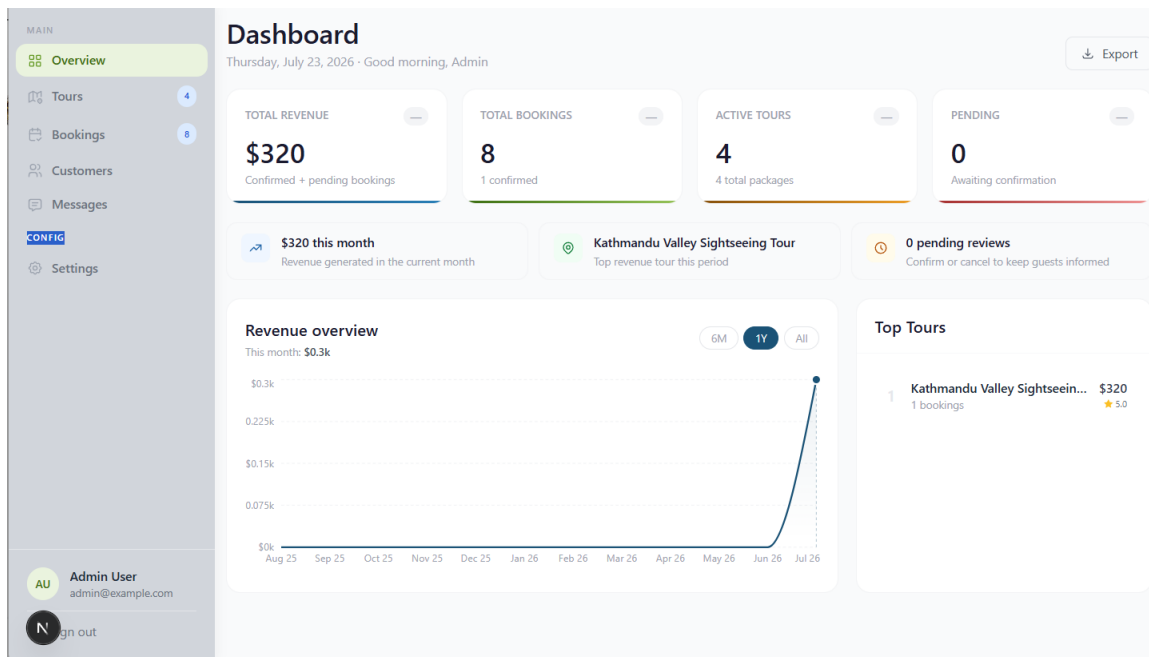
admin@nepaltreks.com

PASSWORD

Sign in

© 2026 Nepal Treks. All rights reserved.

- Admin Dashboard



- Admin Tours Page

Overview

Tours 4

Bookings 1

Customers

Messages

CONF

Settings

Admin User

admin@example.com

Sign out

All Packages

Manage and publish your tour packages

4 TOTAL

4 ACTIVE

0 INACTIVE

All

Trekking

Culture

Adventure

Add Tour

Search...

Rows per page: 10

TOUR NAME	CATEGORY	DURATION	DIFFICULTY	PRICE	RATING	STATUS	ACTIONS
Kathmandu Valley Sightseeing Tour Kathmandu, Nepal	CULTURAL	1 days	EASY	\$55	5.0	ACTIVE	Edit Delete
Chitlang Visit Chitlang	CULTURAL	2 days	EASY	\$100	5.0	ACTIVE	Edit Delete
Langtang Trek Langtang, Nepal	TREKKING	8 days	MODERATE	\$250	5.0	ACTIVE	Edit Delete
Everest Base Camp Everest region, Nepal	TREKKING	14 days	HARD	\$1,300	5.0	ACTIVE	Edit Delete

Showing 1 - 4 of 4 results

- Admin – Add New Tours

Add New Tour

Basics

Itinerary

Highlights

Inclusions

Add-ons

Images

Tour Name *

Tagline

Description *

Price (USD) *

0

Duration (days) *

1

Location *

Tour tags (eg : Family friendly)

Difficulty *

Moderate

Category *

TREKKING

Image URL

Rating (0-5)

5

Review Count

0

Add New Tour

✕

Basics

Itinerary (8)

Highlights (4)

Inclusions (12)

Add-ons (3)

Images (2)

HIGHLIGHT 1

✕

Icon

Title



Explore the ancient walled city of Lo Manthang

Description

HIGHLIGHT 2

✕

Icon

Title



Visit centuries-old Buddhist monasteries

Description

Add New Tour

✕

Basics •

Itinerary (8)

Highlights (4)

Inclusions (12)

Add-ons (3)

Images (2)

✓ INCLUDED

✗ NOT INCLUDED

Airport pickup and drop-off

✕

Tourist bus transportation

✕

Domestic flights (Pokhara-Jomsom-Pokhara)

✕

7 nights accommodation

✕

Daily breakfast, lunch, and dinner

✕

Professional English-speaking guide

✕

+ Add Item

International airfare

✕

Nepal entry visa

✕

Personal expenses

✕

Personal trekking equipment

✕

Laundry and internet charges

✕

Alcoholic and soft drinks

✕

+ Add Item

Cancel

Create Tour

Add New Tour

✕

Basics

Itinerary (8)

Highlights (4)

Inclusions (12)

Add-ons (3)

Images (2)

ADD-ON 1

✕

Name

Porter Service

Price (\$)

60

⬆

⬆

Description

ADD-ON 2

✕

Name

Private Jeep Upgrade

Price (\$)

120

⬆

⬆

Description

- Admin Bookings Page

Bookings

All tour bookings and reservation details.

8 TOTAL BOOKINGS 1 CONFIRMED 0 PENDING \$320 REVENUE

Search...

Rows per page: 10

#	CUSTOMER	TOUR	BOOKED ON	PEOPLE	AMOUNT	PAYMENT	RECEIPT	STATUS
#0008	SR Shovit Regmi regmishovit12@gmail.com	Kathmandu Valley Sightseeing ...	Jul 23, 2026	2	\$320 +\$210 add-ons	eSewa	View	Confirmed
#0007	SP Saugat Panta saugatpant31@gmail.com	Everest Base Camp	Jul 17, 2026	1	\$1,315 +\$15 add-ons	eSewa	View	Cancelled
#0006	SA Sudhan Acharya sudhanacharya21@gmail.com	Chitlang Visit	Jun 12, 2026	2	\$400	eSewa	View	Cancelled
#0005	RA Raman Achhami achhamianar57@gmail.com	—	Jun 11, 2026	3	\$540 +\$240 add-ons	eSewa	View	Cancelled
#0004	SP Saugat Panta sugantapant301@gmail.com	—	Jun 11, 2026	1	\$68	Khalti	View	Cancelled
#0003	SP Saugat Panta sugantapant301@gmail.com	—	Jun 11, 2026	1	\$68	eSewa	View	Cancelled
#0002	SP Saugat Panta	—	Jun 11, 2026	1	\$68	eSewa	View	Cancelled
#0001	SP Saugat Panta	—	Jun 11, 2026	1	\$68	eSewa	View	Cancelled

Admin User
admin@example.com
Sign out

- Admin Customer Details

SR Shovit Regmi
regmishovit12@gmail.com

Confirmed Booking #0008

TOUR DETAILS

TOUR
Kathmandu Valley Sightseeing Tour

DURATION
—

TRAVELERS
2 people

TRAVELER INFO

FULL NAME
Shovit Regmi

EMAIL
regmishovit12@gmail.com

PHONE
+9779866599038

COUNTRY
Nepal

PAYMENT

METHOD
eSewa

Tour price \$55

Add-ons +\$210

Total \$320

Receipt View PDF

UPDATE STATUS

Confirmed Save

- Admin Customers Page

Customers
Manage and view customer information.

5 TOTAL CUSTOMERS | 1 REPEAT BOOKERS | 3 COUNTRIES | \$3,473 REVENUE

Search...

CUSTOMER	COUNTRY	BOOKINGS	TOTAL SPENT	LAST BOOKING	JOINED
SP Saugat Panta saugatpanta31@gmail.com	Nepal	1	\$1,315	Jul 17, 2026	Jul 17, 2026
SP Saugat Panta sumanbasnet301@gmail.com	Nepal	4	\$898	Jun 11, 2026	Jun 11, 2026
RA Raman Achhami achhamiraman07@gmail.com	South Africa	1	\$540	Jun 11, 2026	Jun 11, 2026
SA Sudhan Acharya sudhanacharya21@gmail.com	Moon	1	\$400	Jun 12, 2026	Jun 12, 2026
SR Shovit Regmi regmishovit12@gmail.com	Nepal	1	\$320	Jul 23, 2026	Jul 23, 2026

Showing 1 - 5 of 5 results

- Admin Messages Page

Messages 1 NEW
Contact form submissions from your website.

1 TOTAL | 1 UNREAD | 0 READ

Search...

NAME	EMAIL	SUBJECT	PREVIEW	RECEIVED	ACTIONS
Shovit	regmishovit12@gmail.com	CUSTOM ITINERARY RE...	Hi, can we spend like an entire night...	Jul 23, 2026	

Showing 1 - 1 of 1 results

- Contact Us

Send Us a Message

Fill in the form below and we'll be in touch.

YOUR NAME *

Jane Doe

EMAIL ADDRESS *

jane@example.com

SUBJECT

Select a topic...

MESSAGE *

Tell us about your dream trek, travel dates, group size, or anything else on your mind...

Send Message

Your information is kept private and never shared.

FIND US

Thamel, Kathmandu 44600, Nepal

Near the Thamel Chowk intersection

CALL US

+977 1 4701234

Mon-Sat, 9am-6pm NST

EMAIL US

hello@ebookingnepal.com

We reply within 24 hours

WHATSAPP

LIVE CHAT

+977 98454 39816

Tap to chat — we reply fast

Thamel, Kathmandu

Open in Maps

APPENDIX B: CODE SNIPPETS

Snippet 1: Content-Based Recommendation Engine — Cosine Similarity

```
import { Package } from 'src/entities/package.entity';

export interface ScoredPackage {

  package: Package;

  score: number;

}

function range(values: number[]): { min: number; max: number } {

  if (values.length === 0) return { min: 0, max: 0 };

  return { min: Math.min(...values), max: Math.max(...values) };

}

function normalize(value: number, r: { min: number; max: number }): number {

  if (r.max === r.min) return 0;

  return (value - r.min) / (r.max - r.min);

}

function uniqueValues(packages: Package[], select: (p: Package) => string | undefined):
string[] {

  return Array.from(

    new Set(packages.map((p) => (select(p) ?? "").trim().toLowerCase()).filter(Boolean)),

  );

}

function oneHot(value: string | undefined, universe: string[]): number[] {

  const v = (value ?? "").trim().toLowerCase();

  return universe.map((u) => (u === v ? 1 : 0));

}
```

```

export function buildFeatureVectors(packages: Package[]): Map<number, number[]> {
  const difficulties = uniqueValues(packages, (p) => p.difficulty);
  const categories = uniqueValues(packages, (p) => p.category);

  const priceRange = range(packages.map((p) => Number(p.price)));
  const daysRange = range(packages.map((p) => Number(p.days)));

  const vectors = new Map<number, number[]>();

  for (const p of packages) {
    const vec: number[] = [
      normalize(Number(p.price), priceRange),
      normalize(Number(p.days), daysRange),
      ...oneHot(p.difficulty, difficulties).map((v) => v * 2.0),
      // Category groups tours by type (Trekking vs Sightseeing, etc.).
      ...oneHot(p.category, categories).map((v) => v * 1.5),
    ];
    vectors.set(p.id, vec);
  }
  return vectors;
}

export function cosineSimilarity(a: number[], b: number[]): number {
  let dot = 0;
  let magA = 0;
  let magB = 0;

```

```

for (let i = 0; i < a.length; i++) {
  dot += a[i] * b[i];
  magA += a[i] * a[i];
  magB += b[i] * b[i];
}
if (magA === 0 || magB === 0) return 0;
return dot / (Math.sqrt(magA) * Math.sqrt(magB));
}

export function rankSimilarPackages(
  target: Package,
  candidates: Package[],
  limit = 3,
  minScore = 0.35,
): ScoredPackage[] {
  const others = candidates.filter((c) => c.id !== target.id);
  const vectors = buildFeatureVectors([target, ...others]);
  const targetVec = vectors.get(target.id)!;

  const scored: ScoredPackage[] = others.map((c) => ({
    package: c,
    score: cosineSimilarity(targetVec, vectors.get(c.id)!),
  }));

  return scored
    .filter((s) => s.score >= minScore)

```



```

    .sort((a, b) => b.score - a.score)

    .slice(0, limit);
}

```

Snippet 2: Information Retrieval — TF-IDF Weighted Search Ranking (with partial match)

```

import { Package } from 'src/entities/package.entity';

export interface RankedPackage {

  package: Package;

  score: number;

}

const STOPWORDS = new Set([

  'the', 'a', 'an', 'of', 'in', 'on', 'and', 'to', 'for',

  'with', 'is', 'at', 'from', 'by', 'this', 'that', 'it',

]);

// Match strength multipliers — exact beats stemmed beats partial beats fuzzy.

const MATCH_WEIGHT = {

  exact: 1.0,

  stemmed: 0.85,

  prefix: 0.6,

  substring: 0.45,

  fuzzy: 0.3,

} as const;

// Field weights — a hit in the name/tagline is a stronger relevance signal

// than the same word buried in a long description.

```

```

const FIELD_WEIGHTS: Array<{ field: keyof Package; weight: number }> = [
  { field: 'name', weight: 3 },
  { field: 'tagline', weight: 2.5 },
  { field: 'location', weight: 2 },
  { field: 'category', weight: 1.5 },
  { field: 'difficulty', weight: 1.5 },
  { field: 'description', weight: 1 },
];

function tokenize(text: string): string[] {
  return (text || "")
    .toLowerCase()
    .replace(/[^a-z0-9\s]/g, ' ')
    .split(/\s+/)
    .filter((t) => t.length > 0 && !STOPWORDS.has(t));
}

function stem(term: string): string {
  if (term.length <= 3) return term;
  if (term.endsWith('ies')) return term.slice(0, -3) + 'y';
  if (term.endsWith('ing') && term.length > 5) return term.slice(0, -3);
  if (term.endsWith('ed') && term.length > 4) return term.slice(0, -2);
  if (term.endsWith('es') && term.length > 4) return term.slice(0, -2);
  if (term.endsWith('s') && !term.endsWith('ss')) return term.slice(0, -1);
  return term;
}

function levenshtein(a: string, b: string): number {

```

```

const dp: number[][] = Array.from({ length: a.length + 1 }, (_, i) =>
  Array(b.length + 1).fill(0).map((_, j) => (i === 0 ? j : 0)),
);

for (let j = 0; j <= b.length; j++) dp[0][j] = j;

for (let i = 1; i <= a.length; i++) {
  for (let j = 1; j <= b.length; j++) {
    dp[i][j] =
      a[i - 1] === b[j - 1]
        ? dp[i - 1][j - 1]
        : 1 + Math.min(dp[i - 1][j - 1], dp[i - 1][j], dp[i][j - 1]);
  }
}

return dp[a.length][b.length];
}

interface WeightedDoc {
  tokens: string[]; // raw tokens, field-repeated by weight (for TF)
  stemmedTokens: string[]; // stemmed, same order/weighting
  uniqueTokens: Set<string>;
  uniqueStemmed: Set<string>;
}

function buildWeightedDocument(p: Package): WeightedDoc {
  const tokens: string[] = [];

  for (const { field, weight } of FIELD_WEIGHTS) {
    const raw = (p as any)[field] as string | undefined;
    if (!raw) continue;

```

```

const fieldTokens = tokenize(raw);

// Repeat tokens `weight` times (rounded) so TF naturally reflects field
// importance without needing a separate weighting pass later.

const repeats = Math.max(1, Math.round(weight));

for (let i = 0; i < repeats; i++) tokens.push(...fieldTokens);
}

const stemmedTokens = tokens.map(stem);

return {
  tokens,
  stemmedTokens,
  uniqueTokens: new Set(tokens),
  uniqueStemmed: new Set(stemmedTokens),
};
}

export function rankByKeyword(packages: Package[], keyword: string): RankedPackage[] {
  const rawQueryTerms = tokenize(keyword);

  if (rawQueryTerms.length === 0) {
    return packages.map((p) => ({ package: p, score: 0 }));
  }

  const queryTerms = Array.from(new Set(rawQueryTerms));
  const stemmedQueryTerms = queryTerms.map(stem);
  const docs = packages.map(buildWeightedDocument);
  const N = docs.length;

  // Document frequency (for IDF) computed on stemmed terms so "trek"/"treks"
  // don't get treated as separate, rarer terms than they really are.

```

```

const documentFrequency = new Map<string, number>();
for (const term of stemmedQueryTerms) {
    const count = docs.filter((d) => d.uniqueStemmed.has(term)).length;
    documentFrequency.set(term, count);
}

const idf = new Map<string, number>();
for (const term of stemmedQueryTerms) {
    const df = documentFrequency.get(term) ?? 0;
    idf.set(term, Math.log((N + 1) / (df + 1)) + 1);
}

const scored: RankedPackage[] = packages.map((p, idx) => {
    const doc = docs[idx];

    if (doc.tokens.length === 0) return { package: p, score: 0 };

    const termCounts = new Map<string, number>();

    for (const t of doc.tokens) termCounts.set(t, (termCounts.get(t) ?? 0) + 1);

    const stemmedCounts = new Map<string, number>();

    for (const t of doc.stemmedTokens) stemmedCounts.set(t, (stemmedCounts.get(t) ?? 0) + 1);

    let score = 0;

    for (let i = 0; i < queryTerms.length; i++) {
        const term = queryTerms[i];

        const stemmedTerm = stemmedQueryTerms[i];

        const weightForTerm = idf.get(stemmedTerm) ?? 1;

        // 1. Exact token match.

        if (termCounts.has(term)) {

```

```

const tf = (termCounts.get(term) ?? 0) / doc.tokens.length;

score += tf * weightForTerm * MATCH_WEIGHT.exact;

continue;
}

// 2. Stemmed match ("trekking" query hits "trek" in doc, or vice versa).
if (stemmedCounts.has(stemmedTerm)) {

  const tf = (stemmedCounts.get(stemmedTerm) ?? 0) / doc.stemmedTokens.length;

  score += tf * weightForTerm * MATCH_WEIGHT.stemmed;

  continue;

}

// 3. Prefix / substring partial credit against unique doc tokens.
let bestPartial = 0;

let matchedPartial = false;

for (const docToken of doc.uniqueTokens) {

  if (docToken.length < 3 || term.length < 3) continue;

  if (docToken.startsWith(term) || term.startsWith(docToken)) {

    bestPartial = Math.max(bestPartial, MATCH_WEIGHT.prefix);

    matchedPartial = true;

  } else if (docToken.includes(term) || term.includes(docToken)) {

    bestPartial = Math.max(bestPartial, MATCH_WEIGHT.substring);

    matchedPartial = true;

  }

}

if (matchedPartial) {

  score += weightForTerm * bestPartial * 0.3; // no true TF here, flat partial credit

```

```

    continue;
  }

  // 4. Fuzzy fallback for typos — only for reasonably long terms, to
  // avoid noisy matches on short words.
  if (term.length >= 4) {
    const maxDist = term.length <= 5 ? 1 : 2;

    for (const docToken of doc.uniqueTokens) {
      if (Math.abs(docToken.length - term.length) > maxDist) continue;
      if (levenshtein(term, docToken) <= maxDist) {
        score += weightForTerm * MATCH_WEIGHT.fuzzy * 0.3;
        break;
      }
    }
  }

  return { package: p, score };
});

return scored.filter((s) => s.score > 0).sort((a, b) => b.score - a.score);
}

```

